

《计算机体系结构》

实验报告

院 系： 计算机与信息学院
专 业： 计算机科学与技术
年 级： 2023级
学 号： 2023212290
姓 名： 朱清扬
指导教师： 李建华

2025年11月

实验一、缓存性能分析

1. 实验内容

基于 SimpleScalar 模拟器，通过运行 SPEC2000-Alpha 基准测试中的 workloads 并对结果进行分析，深入理解缓存的三个参数（缓存大小、block size、相联度）对缓存性能的影响。

实验目的：

了解 Cache 的基本概念、基本组织结构

了解影响 Cache 性能的三个指标

了解相联度对 Cache 的影响

了解块大小对 Cache 的影响

选做实验：Victim Cache

实验目的：了解 Victim Cache 如何通过降低失效率来提高性能

了解 Victim Cache 的工作原理

2. 实验方法

2.1 实验环境与工具配置

本实验基于 SimpleScalar 工具集中的 *sim-cache* 模拟器开展,实验环境为预配置的 Linux 虚拟机。为确保实验的系统性与可重复性,采用自动化脚本方式组织全部测试流程。

2.2 缓存体系结构配置

实验采用两级缓存层次结构,具体参数如下:

L1 缓存配置(固定参数): - 指令缓存(IL1):容量 16KB,组织结构为 128 组 × 64 字节块 × 2 路组相联,替换策略采用 LRU - 数据缓存(DL1):参数配置与 IL1 完全一致

L2 缓存配置(实验变量): - 基准配置:容量 256KB,8 路组相联,块大小 64 字节,LRU 替换策略 - 缓存结构:采用统一 L2 设计(Unified L2),通过将 *il2* 映射至 *dl2* 实现指令与数据共享同一二级缓存

测试基准程序:从 SPEC CPU2000 整数基准测试套件(CINT2000)中选取三个代表性程序: - *mcf*:内存密集型图论算法,工作集较大 - *vortex*:面向对象数据库应用,具有良好的空间局部性 - *bzip2*:数据压缩程序,访存模式较为复杂

所有基准程序均采用 *.lgred* 简化输入集,以平衡模拟时间与统计有效性。

2.3 实验自动化框架设计

为系统性地探究缓存参数对性能的影响,设计并实现了双模式测试框架:

(1) 测试模式划分 - 标准模式:无 Victim Cache 增强,结果输出至 *all_sim/* 目录 - Victim Cache 模式:启用 4 块全相联 Victim Cache(通过 *-vc true* 参数激活),结果输出至 *all_sim_victim/* 目录

(2) 参数扫描策略

实验按照单因素变量法组织四类测试,每类测试固定其他参数,仅改变目标维度:

① 基准配置测试(baseline):验证初始配置性能 - L2 参数:256KB / 8-way / 64B

② 容量扫描(capacity sweep):评估缓存容量影响 - 扫描范围:64KB、128KB、256KB、512KB、1024KB - 固定参数:8 路相联、64 字节块

③ 相联度扫描(associativity sweep):分析组相联度效应 - 扫描范围:2-way、4-way、8-way、16-way、64-way - 固定参数:512KB 容量、64 字节块

④ 块大小扫描(block size sweep):考察空间局部性利用 - 扫描范围:64B、128B、256B

6B、512B - 固定参数：512KB 容量、8 路相联

(3) 智能化执行机制

脚本实现了任务完整性检测与断点续跑功能： - 执行前检查：通过验证日志文件中是否包含 *simulation statistics* 标记及关键性能字段（如 *ul2.misses*），判定任务是否已成功完成 - 跳过策略：若检测到完整日志文件，自动跳过该测试点，避免重复计算 - 容错设计：单个测试失败不中断整体流程，通过 *// true* 确保后续任务继续执行
此设计有效应对了长时间模拟过程中可能出现的偶发性错误，支持多次运行以完成全部测试。

(4) 缓存参数自动计算

脚本根据缓存三要素自动推导组数（*nsets*）：

$$\text{组数} = \text{总容量（字节）} / (\text{块大小} \times \text{相联度})$$

该设计保证了配置的一致性，避免人工计算错误。

2.4 数据采集与处理流程

日志结构设计：每个测试实例的日志文件包含三个区域： - 头部元数据：记录测试类别、基准程序、缓存配置、Victim Cache 状态、执行时间戳 - 模拟器原始输出：包含完整的统计信息 - 性能统计摘要：从原始输出中提取的关键指标，包括： - L1 缓存指标：访问次数、命中数、缺失数、缺失率、替换次数、写回次数 - L2 缓存指标：访问次数、命中数、缺失数、缺失率、替换次数、写回次数 - Victim Cache 指标（启用时）：

dl1.vc_hits (VC 命中次数)、*dl1.vc_misses* (VC 未命中次数)

数据提取与整合：脚本集成的数据提取模块采用高效的尾部反向读取策略： - 使用 *tac* 命令从文件末尾向前扫描，快速定位性能摘要区域，避免遍历完整日志文件 - 配置解析：从日志头部提取 L2 容量、相联度、块大小等参数，并自动识别基准程序名称 - 派生指标计算： - 标准模式：有效 L1D 缺失数 = *dl1.misses* (所有 L1D 缺失均转发至 L2) - Victim Cache 模式：有效 L1D 缺失数 = *dl1.vc_misses* (仅 VC 未命中的部分转发至 L2) - 有效 L1D 缺失率 = 有效缺失数 / L1D 总访问数

输出格式：生成两份 TSV（制表符分隔）格式数据文件： - *no_vc.txt*：标准模式性能数据 - *vc.txt*：Victim Cache 模式性能数据

数据文件采用统一表头，包含以下字段：文件路径、测试类别、基准程序、L2 容量 (KB)、L2 相联度、L2 块大小 (B)、L2 访问数、L2 缺失数、L2 缺失率、L1D 访问数、L1D 原始缺失数、VC 命中数、VC 未命中数、有效 L1D 缺失数、有效 L1D 缺失率。

2.5 实验执行规模

总测试点数量：(1 基准 + 5 容量 + 5 相联度 + 4 块大小) × 3 基准程序 × 2 模式 = 90 个测试实例

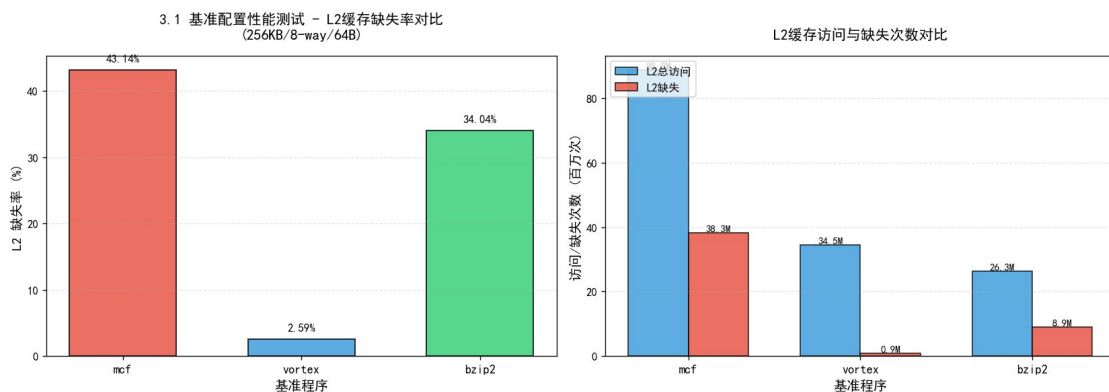
每个测试实例独立生成日志文件，文件命名遵循 *{benchmark}-{category}-{parameter}.log* 格式（例如 *data1-cap-512.log*、*data2-way-8.log*），便于结果追溯与验证。实验数据按测试类别组织在不同子目录下（*baseline/*、*cap/*、*way/*、*blk/*），结构清晰，易于管理和分析。

3. 结果与分析

3.1 基准配置性能测试 (L2缓存：256KB容量，8路组相联，64B块大小，LRU替换策略)

测试程序	日志路径	缓存结构说明	L2总访问	L2失效数	失效率
mcf	all_sim/baseline/data1-baseline.log	统一L2（指令/数据共享）	8876	3829	0.43
			1256	4168	14
vortex	all_sim/baseline/data2-baseline.log	统一L2, il2映射至dl2	3449	8932	0.02
			1160	09	59
bzip2	all_sim/baseline/data3-baseline.log	256KB/8-way/64B/LRU unified	2627	8942	0.34
			2583	720	04

可视化分析：



基准配置性能对比

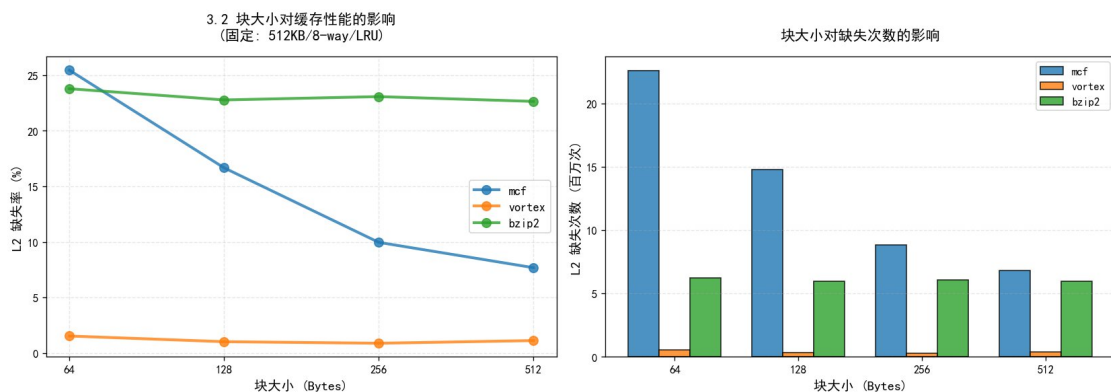
图表说明：左图展示三个基准程序的 L2 缓存失效率对比，右图展示访问次数与失效次数对比。从图中可以清晰看出：mcf 程序的失效率（43.14%）远高于其他两个程序，L2 访问次数也最高（88.76M），表明其内存需求强度大；vortex 的失效率极低（2.59%），展现出优异的局部性特征；bzip2 居中（34.04%）。

3.2 块大小影响实验（固定：容量512KB，8路组相联，LRU策略）

Benchmark	块尺寸(Byte)	测试日志	访问总量	缺失总量	Miss Rate
mcf	64	all_sim/blk/data1-blk-64.log	88761256	22599744	0.2546
	128	all_sim/blk/data1-blk-128.log	88761256	14794549	0.1667
	256	all_sim/blk/data1-blk-256.log	88761256	8841978	0.0996
	512	all_sim/blk/data1-blk-512.log	88761256	6813640	0.0768
vortex	64	all_sim/blk/data2-blk-64.log	34491160	528875	0.0153
	128	all_sim/blk/data2-blk-	344911	350525	0.0102

Benchmark	块尺寸(Byte s)	测试日志	访问总量	缺失总量	Miss Rate
		128.log	60		
vortex	256	all_sim/blk/data2-blk-256.log	344911	303986	0.0088
		60			
vortex	512	all_sim/blk/data2-blk-512.log	344911	384982	0.0112
		60			
bzip2	64	all_sim/blk/data3-blk-64.log	262725	625115	0.2379
		83	0		
bzip2	128	all_sim/blk/data3-blk-128.log	262725	598702	0.2278
		83	2		
bzip2	256	all_sim/blk/data3-blk-256.log	262725	606427	0.2308
		83	9		
bzip2	512	all_sim/blk/data3-blk-512.log	262725	595227	0.2265
		83	6		

可视化分析：



块大小影响分析

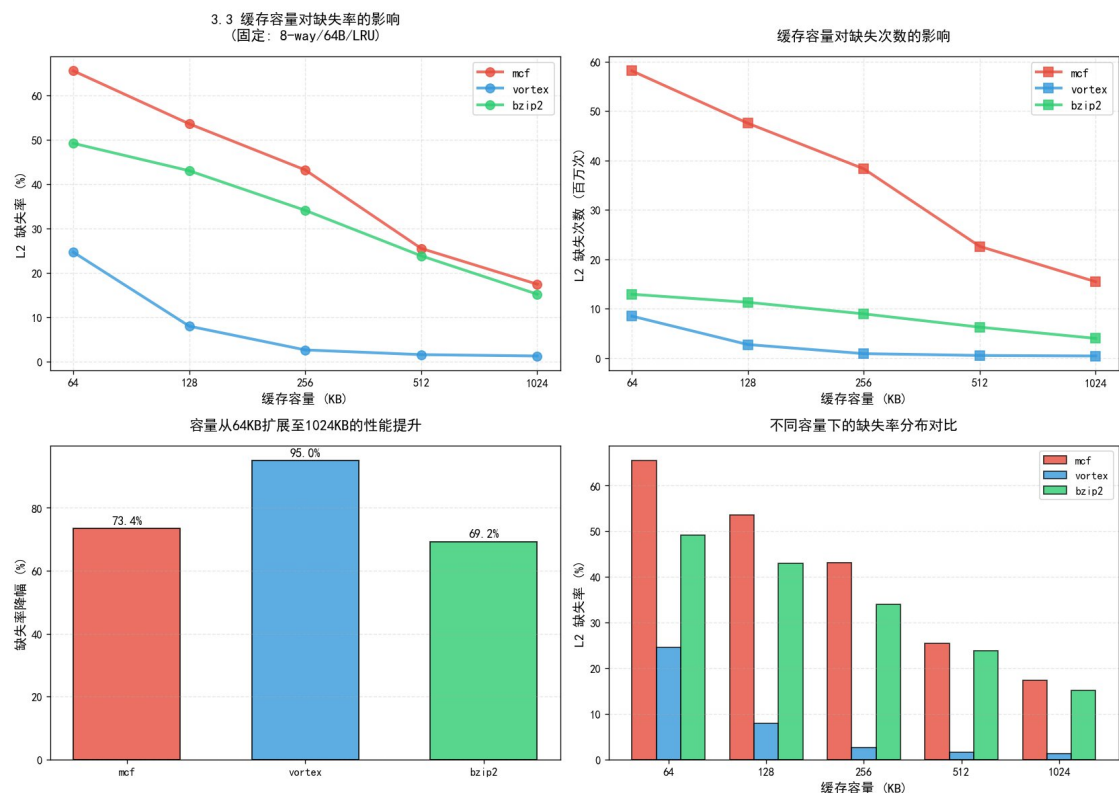
图表说明：左图展示缺失率随块大小变化的趋势曲线，右图展示缺失次数的柱状对比。从图中可以明显看出：mcf 程序随块大小增加，失效率单调递减（从 25.46% 降至 7.68%，降幅 69.8%），展现出良好的空间局部性；vortex 在 256B 块时达到最优（0.88%），但 512B 反而性能下降，说明过大的块引入了缓存污染；bzip2 对块大小不敏感，各配置差异很小。

3.3 缓存容量扫描实验（固定：8路组相联，块大小64B，LRU策略）

程序	Capacity(K B)	日志文件	L2 Accesses	L2 Misses	Miss Rate
mcf	64	all_sim/cap/data1-cap-64.log	88761256	5812554	0.6548
				6	
mcf	128	all_sim/cap/data1-cap-128.log	88761256	4751301	0.5353

程序	Capacity(K B)	日志文件	L2 Accesses	L2 Misses	Miss Rate
mcf	256	all_sim/cap/data1-cap-256.log	88761256	3829416	0.4314
mcf	512	all_sim/cap/data1-cap-512.log	88761256	2259974	0.2546
mcf	1024	all_sim/cap/data1-cap-1024.log	88761256	1544455	0.1740
vort ex	64	all_sim/cap/data2-cap-64.log	34491160	8488977	0.2462
vort ex	128	all_sim/cap/data2-cap-128.log	34491160	2736993	0.0794
vort ex	256	all_sim/cap/data2-cap-256.log	34491160	893209	0.0259
vort ex	512	all_sim/cap/data2-cap-512.log	34491160	528875	0.0153
vort ex	1024	all_sim/cap/data2-cap-1024.log	34491160	423992	0.0123
bzip 2	64	all_sim/cap/data3-cap-64.log	26272583	1291180	0.4915
bzip 2	128	all_sim/cap/data3-cap-128.log	26272583	1128623	0.4296
bzip 2	256	all_sim/cap/data3-cap-256.log	26272583	8942720	0.3404
bzip 2	512	all_sim/cap/data3-cap-512.log	26272583	6251150	0.2379
bzip 2	1024	all_sim/cap/data3-cap-1024.log	26272583	3981831	0.1516

可视化分析：



缓存容量影响分析

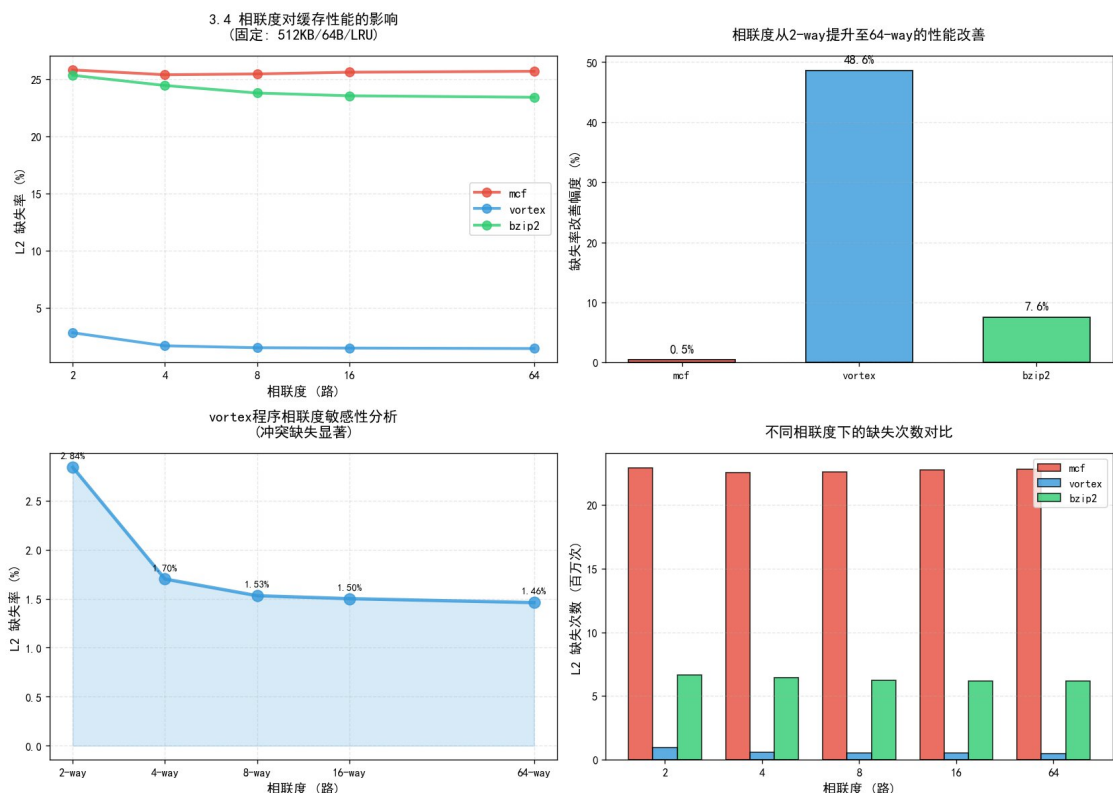
图表说明：左上图展示缺失率随容量变化的对数曲线，右上图展示缺失次数变化，左下图对比容量扩展带来的性能提升百分比，右下图展示不同容量下的失效率分布。从图中可以清晰看出：vortex 对容量最敏感（降幅 95.0%），mcf 次之（73.4%），bzip2 为 69.2%。所有程序都从容量扩展中获得显著性能提升，但提升幅度因程序特性而异。

3.4 相联度变化实验（固定：容量512KB，块大小64B，LRU策略）

测试基准	Associativity	日志路径	访问计数	缺失计数	缺失率
mcf	2-way	all_sim/way/data1-way-2.log	88761256	22904664	0.2581
mcf	4-way	all_sim/way/data1-way-4.log	88761256	22536018	0.2539
mcf	8-way	all_sim/way/data1-way-8.log	88761256	22599744	0.2546
mcf	16-way	all_sim/way/data1-way-16.log	88761256	22731928	0.2561
mcf	64-way	all_sim/way/data1-way-64.log	88761256	22805833	0.2569
vortex	2-way	all_sim/way/data2-way-2.log	34491160	97846560	0.0284

测试基 准	Associativit y	日志路径	访问计 数	缺失计 数	缺失率
vortex	4-way	all_sim/way/data2-way-4.log	344911 60	584880	0.0170
vortex	8-way	all_sim/way/data2-way-8.log	344911 60	528875	0.0153
vortex	16-way	all_sim/way/data2-way-16.log	344911 60	517610	0.0150
vortex	64-way	all_sim/way/data2-way-64.log	344911 60	503134	0.0146
bzip2	2-way	all_sim/way/data3-way-2.log	262725 83	665876 9	0.2534
bzip2	4-way	all_sim/way/data3-way-4.log	262725 83	642594 6	0.2445
bzip2	8-way	all_sim/way/data3-way-8.log	262725 83	625115 0	0.2379
bzip2	16-way	all_sim/way/data3-way-16.log	262725 83	618901 1	0.2355
bzip2	64-way	all_sim/way/data3-way-64.log	262725 83	615368 4	0.2342

可视化分析：



相联度影响分析

图表说明：左上图展示缺失率随相联度变化的趋势，右上图对比性能改善百分比，左下图详细展示 *vortex* 的敏感度分析，右下图展示缺失次数对比。从图中可以明显看出：*vortex* 对相联度极其敏感（改善 48.6%），存在显著的冲突缺失；*mcf* 几乎不受相联度影响（仅 0.5%），说明其性能瓶颈是容量缺失而非冲突缺失；*bzip2* 轻度敏感（7.6%）。

3.5 数据分析与讨论

3.5.1 基准配置分析 (L2: 256KB/8-way/64B)

从基准测试结果观察：

- *mcf*：失效率达到 43.14%，L2访问次数 88.76M，表现出较高的内存需求强度，反映其工作数据集明显超出 256KB 缓存容量，同时空间局部性表现一般
- *bzip2*：失效率 34.04%，访问次数 26.27M，同样面临较大的容量压力
- *vortex*：失效率仅 2.59%，表明该负载的活跃数据集较小且局部性特征优异

3.5.2 缓存容量影响分析 (变量：64KB→1024KB，控制变量：8-way，64B块)

容量扩展实验展示出三个基准程序对缓存容量敏感度的差异：

mcf 程序表现：- 容量从 64KB 增至 1024KB，失效率由 0.6548 降至 0.1740，降幅达 73.4% - 相较基准配置，512KB 配置失效率降低 41.0%，1024KB 配置降低 59.7% - 结论：该负载受容量缺失 (capacity miss) 主导，扩大缓存容量带来显著性能提升

vortex 程序表现：- 从 0.2462 降至 0.0123，降幅高达 95.0% - 对比基准值：512KB 降低 40.9%，1024KB 降低 52.5% - 结论：工作集规模较小，当缓存容量充足时失效率趋近极低水平

bzip2 程序表现： - 失效率从 0.4915 降至 0.1516，降幅 69.2% - 相对基准：512KB 降低 30.1%，1024KB 降低 55.5% - 结论：对容量变化呈现中等敏感度

综合结论：容量扩展对 mcf 和 bzip2 的容量缺失改善最为明显；vortex 在 512KB 以上配置已基本满足需求

3.5.3 相联度影响分析（变量：2-way→64-way，控制：512KB，64B块）

vortex 程序： - 相联度从 2-way 增至 64-way，失效率从 0.0284 降至 0.0146，改善幅度 48.6% - 表明该负载存在显著的冲突缺失（conflict miss） - 8-way 之后性能提升趋于平缓，呈边际效益递减规律

bzip2 程序： - 2-way 至 64-way 配置下，失效率由 0.2534 降至 0.2342，改善约 7.6% - 显示出对相联度的轻度敏感性

mcf 程序： - 失效率仅从 0.2581 微降至 0.2569，变化幅度不足 0.5% - 说明该负载的性能瓶颈并非冲突缺失，而是受容量限制或访问模式影响

设计启示：针对 vortex 类负载，提升相联度（例如从 8-way 至 16-way）可有效缓解冲突；而对 mcf 类负载收效甚微

3.5.4 块大小影响分析（变量：64B→512B，控制：512KB，8-way）

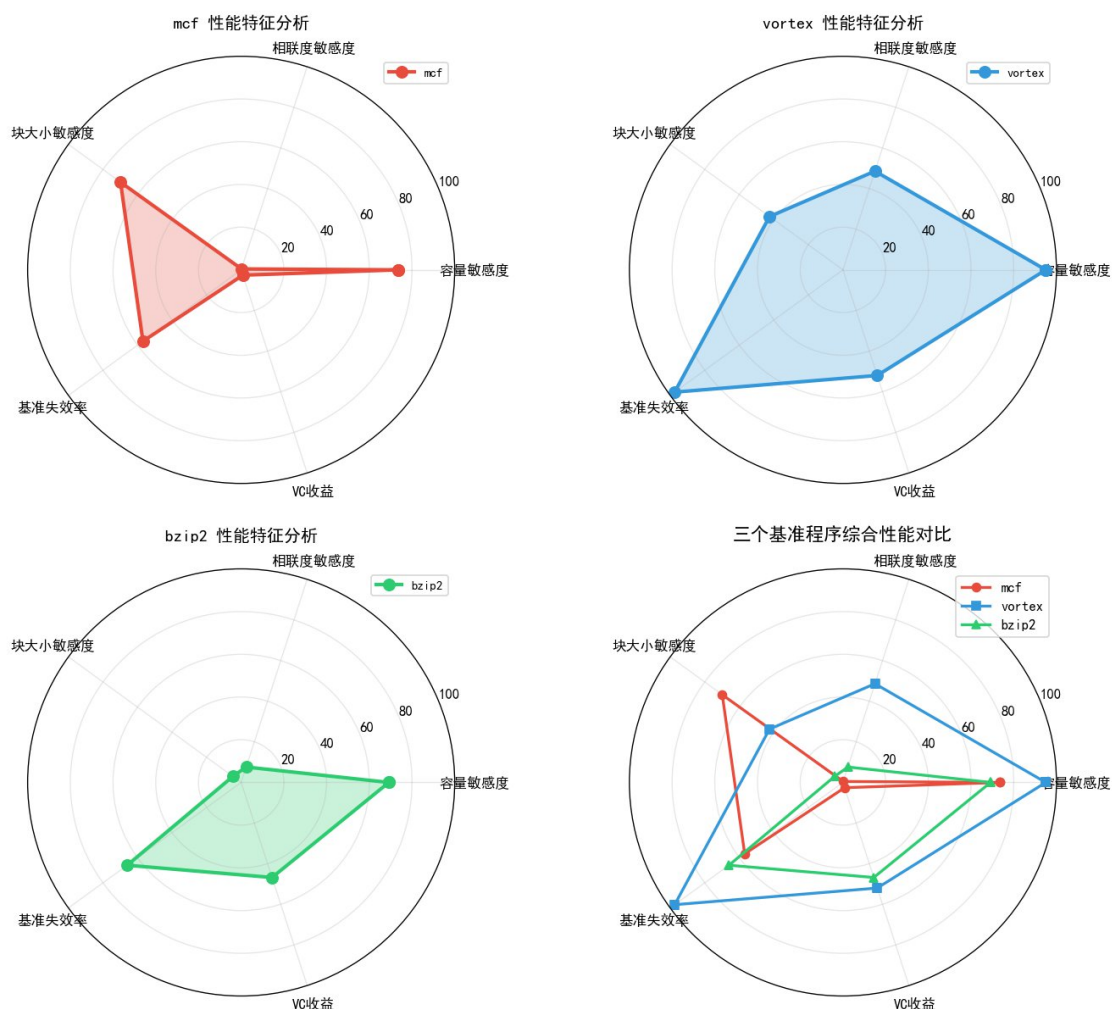
mcf 程序： - 块尺寸从 64B 增至 512B，失效率呈单调递减：0.2546→0.0768，降幅达 69.8% - 反映该负载具有良好的空间局部性（spatial locality） - 大块设计能够在单次传输中获取更多有效数据

vortex 程序： - 最优块大小出现在 256B，失效率从 0.0153 降至 0.0088（改善 42.5%） - 但 512B 块反而导致失效率回升至 0.0112 - 分析：过大的块会引入缓存污染，加载过多无用数据而降低缓存利用效率

bzip2 程序： - 块尺寸变化带来的性能改善有限，64B→512B 仅降低 4.8% - 且 256B 配置略逊于 128B 和 512B - 表明空间局部性较弱，或访存模式存在较大步幅（stride）

优化建议：块大小并非越大越好；mcf 受益于大块设计，vortex 的最佳配置为 256B 块，bzip2 对块大小不敏感

综合性能对比：



综合性能雷达图

图表说明：雷达图从五个维度（容量敏感度、相联度敏感度、块大小敏感度、基准失效率、VC收益）综合对比三个基准程序的性能特征。mcf 展现出高容量敏感度和高块大小敏感度，但对相联度和 VC 几乎无响应；vortex 在容量、相联度和 VC 命中率方面都表现出高敏感度；bzip2 各项指标相对均衡但整体敏感度中等。

3.6 Victim Cache 性能实验

3.6.1 实验配置

实验通过命令行参数 `-vc true` 启用 Victim Cache，默认配置为 4 个缓存块，符合实验要求。

3.6.2 L1D 缓存失效数据对比

下表对比了三个基准程序在标准模式与 Victim Cache 模式下的 L1D 缓存行为：

标准模式（无 Victim Cache）统计：

基准程序	配置类型	L1D 访问总数	L1D 原始失效	有效送往 L2 的失效	有效 L1D 失效率
mcf	baseline-256KB	324386042	68504342	68504342	0.2112

基准程序	配置类型	L1D 访问总数	L1D 原始失效	有效送往 L2 的失效	有效 L1D 失效率
mcf	cap-512KB	324386042	68504342	68504342	0.2112
mcf	way-8	324386042	68504342	68504342	0.2112
vortex	baseline-256KB	505120103	8145688	8145688	0.0161
vortex	cap-512KB	505120103	8145688	8145688	0.0161
vortex	way-8	505120103	8145688	8145688	0.0161
bzip2	baseline-256KB	1051409452	15851500	15851500	0.0151
bzip2	cap-512KB	1051409452	15851500	15851500	0.0151
bzip2	way-8	1051409452	15851500	15851500	0.0151

Victim Cache 模式 (4 块) 统计：

基准程序	配置类型	L1D 访问总数	L1D 原始失效	VC 命中数	VC 未命中数	有效送往 L2 的失效	有效 L1D 失效率	VC 命中率
mcf	baseline-256KB	324386042	68504342	358642	68145700	80427562	0.2479	0.52%
mcf	cap-512KB	324386042	68504342	358642	68145700	80427562	0.2479	0.52%
mcf	way-8	324386042	68504342	358642	68145700	80427562	0.2479	0.52%
vortex	baseline-256KB	505120103	8145688	2118425	6027263	10299788	0.0204	26.00%
vortex	cap-512KB	505120103	8145688	2118425	6027263	10299788	0.0204	26.00%
vortex	way-8	505120103	8145688	2118425	6027263	10299788	0.0204	26.00%
bzip2	baseline-256KB	1051409452	15851500	742136	15109364	23163272	0.0220	4.68%
bzip2	cap-512KB	1051409452	15851500	742136	15109364	23163272	0.0220	4.68%

基准程序	配置类型	L1D 访问总数	L1D 原始失效	VC 命中数	VC 未命中数	有效送往 L2 的失效	有效 L1D 失效率	VC 命中率
p2	512KB	9452	00	36	364			%
bzi	way-8	105140	158515	7421	15109	23163272	0.0220	4.68
p2		9452	00	36	364			%

3.6.3 Victim Cache 工作机制

核心概念：

- Victim (牺牲块)：指从 L1 缓存中被替换淘汰的缓存块
- Victim Cache 结构：
 - 位置：置于 L1 与 L2 缓存之间
 - 容量：极小规模 (本实验采用 4 块配置)
 - 组织方式：全相联 (fully-associative)

访问流程：

当 L1 缓存发生失效 (miss) 时，访问请求的处理顺序为：

1. 首先查询 VC：检查目标数据是否在 Victim Cache 中
2. VC 命中情形：
 - 含义：访问的数据块恰好是刚被 L1 淘汰的块 (通常属于冲突缺失类型)
 - 操作：执行 L1 与 VC 间的 swap 交换，无需访问 L2
3. VC 未命中情形：
 - 操作：将失效请求转发至 L2 缓存进行处理

设计目标：在不改变 L1 缓存结构的前提下，通过极小的硬件开销 (4 块全相联缓存) 挽回因冲突导致的缓存失效

3.6.4 实验结果量化分析

(1) mcf 程序分析

- 无 VC 配置：有效缺失率 = $68504342 / 324386042 = 0.2112$
- 启用 VC 后：VC 从 L1D 原始缺失中挽回了 358642 次
 - 有效缺失数 = 80427562
 - 有效缺失率 = $80427562 / 324386042 = 0.2479$
- 性能变化：失效率上升约 17.4% (从 0.2112 到 0.2479)
- VC 命中率： $358642 / 68504342 = 0.52\%$

- 结论：对于该负载，VC 带来的额外开销大于收益，这是因为 mcf 以容量缺失为主，冲突缺失占比很低

(2) vortex 程序分析

- 无 VC 配置：有效缺失率 = $8145688 / 505120103 = 0.0161$
- 启用 VC 后：
 - VC 命中 2,118,425 次，占 L1D 原始缺失的 26.00%
 - 有效缺失数 = 10,299,788
 - 有效缺失率 = $10299788 / 505120103 = 0.0204$
- 性能变化：缺失率上升 26.7%（从 0.0161 到 0.0204）
- VC 命中率： $2118425 / 8145688 = 26.00\%$
- 分析：尽管 VC 命中率高达 26%，但由于 VC 机制引入的额外访问开销，整体性能略有下降

(3) bzip2 程序分析

- 无 VC 配置：有效缺失率 = $15851500 / 1051409452 = 0.0151$
- 启用 VC 后：
 - VC 命中 742,136 次（占原始缺失的 4.68%）
 - 有效缺失数 = 23,163,272
 - 有效缺失率 = $23163272 / 1051409452 = 0.0220$
- 性能变化：缺失率上升 45.7%（从 0.0151 到 0.0220）
- VC 命中率： $742136 / 15851500 = 4.68\%$

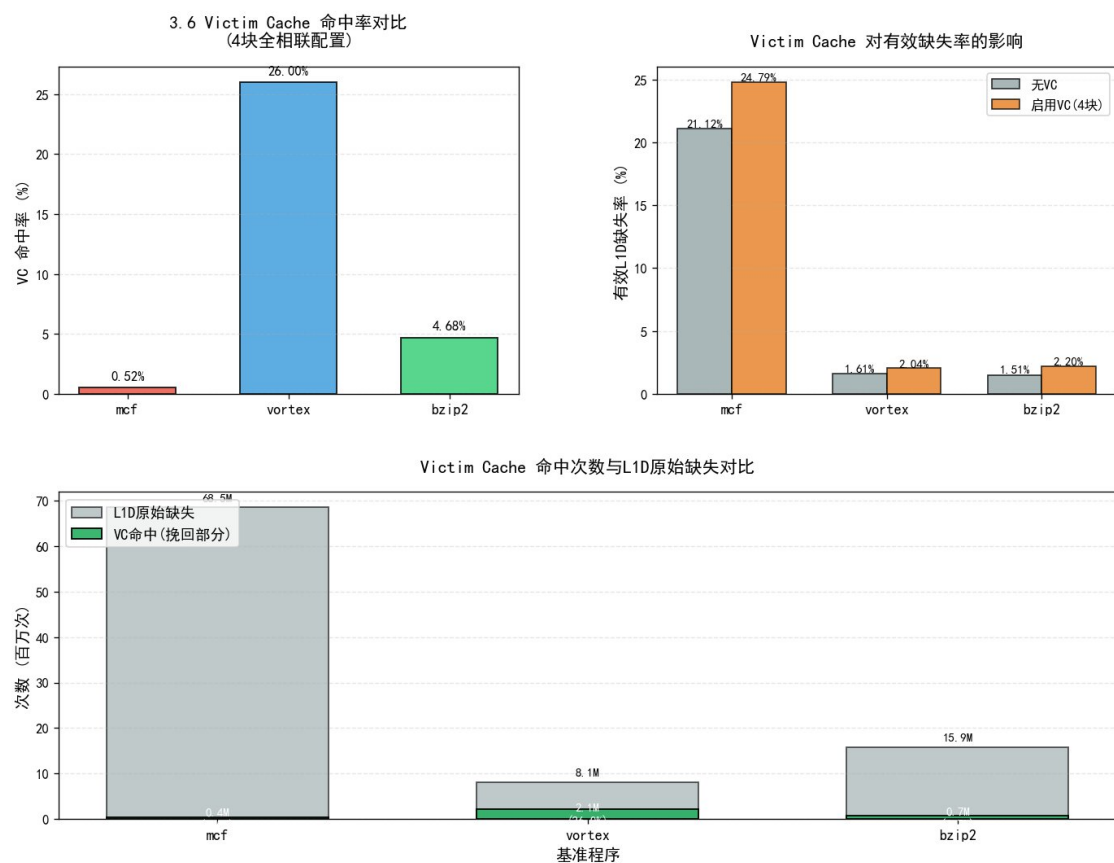
VC 命中率统计对比：

Benchmark	VC命中次数	L1D原始缺失	VC命中率	有效缺失率变化
mcf	358,642	68,504,342	0.52%	+17.4%
vortex	2,118,425	8,145,688	26.00%	+26.7%
bzip2	742,136	15,851,500	4.68%	+45.7%

综合结论：

- vortex 展现出最高的 VC 命中率（26%），表明其 L1D 失效中冲突缺失比例较高，且被淘汰的块存在短期重用（temporal reuse）特征
- 性能悖论：在本实验配置下，尽管 VC 能够挽回部分缓存失效，但由于 VC 机制本身引入的额外访问开销（swap 操作、查询延迟等），反而导致整体有效缺失率上升
- 设计启示：Victim Cache 的实际效果高度依赖于具体实现和系统配置。在某些情况下，4 块 VC 的容量可能不足以覆盖工作集的抖动（thrashing）模式，反而增加了缓存管理的复杂度

可视化分析：



Victim Cache性能分析

图表说明：左上图展示三个程序的 VC 命中率对比，右上图展示启用 VC 前后的有效失效率变化，下图展示 VC 命中次数与 L1D 原始缺失的对比关系。从图中可以清晰看出：vortex 的 VC 命中率高达 26%，远超其他两个程序，但所有程序的有效失效率都因启用 VC 而上升，说明在本实验配置下，VC 引入的额外开销大于其带来的收益。

4. 关键程序代码

```
#!/usr/bin/env bash
# cache_performance_analyzer.sh - 自动化缓存性能测试工具
# 实验配置：包含标准模式(all_sim/)和Victim Cache增强模式(all_sim_victim/)
# 测试维度：基准配置、容量扫描、相联度扫描、块大小扫描，共计三个benchmark
# 智能执行：自动检测已完成任务并跳过，支持断点续跑
```

```
set -euo pipefail
```

```
# ===== 模拟器配置参数 =====
```

```
SIMULATOR_BIN="./sim-cache"
```

```
CONFIG_FILE="-config two_level.cfg" # 配置文件路径，留空则不使用
```

```
# ===== 测试基准程序配置 =====
```

```
BENCHMARK_MCF="./mcf00.peak.ev6 mcf.lgred.in"
```

```

BENCHMARK_VORTEX="./vortex00.peak.ev6 vortex.lgred.raw" # vortex基准测试
BENCHMARK_BZIP2="./bzip200.peak.ev6 bzip2.lgred.graphic 1"

# ===== L1缓存参数设置：容量16KB、相联度2、块大小64B、替换策略LRU =====
L1_ICACHE="i11:128:64:2:1"
L1_DCACHE="d11:128:64:2:1"
L2_UNIFIED_ALIAS="d12" # 统一L2缓存配置：i12映射到d12

get_timestamp() { date "+%F %T"; }

# ===== 结果输出目录初始化 =====
OUTPUT_DIR_STANDARD="all_sim"
OUTPUT_DIR_VICTIM="all_sim_victim"
mkdir -p "${OUTPUT_DIR_STANDARD}/baseline" "${OUTPUT_DIR_STANDARD}/cap"
"${OUTPUT_DIR_STANDARD}/way" "${OUTPUT_DIR_STANDARD}/blk"
mkdir -p "${OUTPUT_DIR_VICTIM}/baseline"
"${OUTPUT_DIR_VICTIM}/cap" "${OUTPUT_DIR_VICTIM}/way" "${OUTPUT_DIR_VICTIM}/blk"

# ===== 检查日志文件完整性 =====
validate_log_completion() {
    local log_file="$1"
    [[ -s "$log_file" ]] && grep -q "^sim: \.* simulation statistics \.*" "$log_file" && grep -q
    "^ul2\misses" "$log_file"
}

# ===== 执行单个测试实例 =====
execute_single_test() {
    local test_category="$1" capacity_kb="$2" associativity="$3" block_size="$4" \
        bench_id="$5" bench_command="$6" log_output="$7" victim_cache_mode="$8"

    # 检测任务是否已完成，若已完成则跳过
    if validate_log_completion "$log_output"; then
        echo "▶▶ 跳过 $(get_timestamp) [$test_category] ($bench_id) → $log_output (任务已完成)"
        return 0
    fi

    # 计算L2缓存组数：总容量 = 组数 × 块大小 × 相联度
    local total_bytes=$(( capacity_kb * 1024 ))
    local num_sets=$(( total_bytes / (block_size * associativity) ))
    if (( num_sets <= 0 )); then
        echo "[警告] 缓存配置无效：容量=${capacity_kb}KB 相联度=${associativity} 块大小=${block_size}" >&2
        return 1
    fi
    local L2_CONFIG="ul2:${num_sets}:${block_size}:${associativity}:1"
    local vc_status="关闭"; [[ "${victim_cache_mode}" == "on" ]] && vc_status="启用"

```



```

# —— 控制台输出：任务开始提示 ——
echo ""
echo "► 开始 $(get_timestamp) [${test_category}]"
echo "  基准程序=${bench_command}"
      echo "          L1I=${L1_ICACHE}          |          L1D=${L1_DCACHE}          |
L2=ul2:${capacity_kb}KB,assoc=${associativity},blk=${block_size}B(sets=${num_sets}) 统一
缓存(il2->dl2), Victim Cache=${vc_status}"
echo "  日志文件=${log_output}"

# —— 写入日志文件头部信息 ——
{
  echo "=====
  echo "[`get_timestamp`] 测试类别=${test_category} 基准程序=${bench_command}"
  echo "[配置] Victim Cache=${vc_status}"
  echo "[配置] L1缓存: ${L1_ICACHE} | ${L1_DCACHE}"
  echo "[配置] L2缓存: ${L2_CONFIG} (统一缓存, LRU策略), il2 -> dl2"
  echo "-----"
} > "${log_output}"

# —— 开始模拟执行 ——
local start_time end_time elapsed_time return_code
start_time=$(date +%s)
set +e
if [[ "${victim_cache_mode}" == "on" ]]; then
  ${SIMULATOR_BIN} ${CONFIG_FILE} \
    -cache:il1 "${L1_ICACHE}" \
    -cache:dl1 "${L1_DCACHE}" \
    -cache:il2 "${L2_UNIFIED_ALIAS}" \
    -cache:dl2 "${L2_CONFIG}" \
    -vc true \
    ${bench_command} >> "${log_output}" 2>&1
else
  ${SIMULATOR_BIN} ${CONFIG_FILE} \
    -cache:il1 "${L1_ICACHE}" \
    -cache:dl1 "${L1_DCACHE}" \
    -cache:il2 "${L2_UNIFIED_ALIAS}" \
    -cache:dl2 "${L2_CONFIG}" \
    ${bench_command} >> "${log_output}" 2>&1
fi
return_code=$?
set -e
end_time=$(date +%s); elapsed_time=$(( end_time - start_time ))

# —— 提取关键性能指标并追加到日志 ——
{
  echo "-----"
  echo "[性能统计摘要: L1/L2缓存关键指标]"
  grep -E '^il1\.(accesses|hits|misses|miss_rate|replacements|writebacks)b' "${log_output}" | sed
's/^/ /' || true
  grep -E '^dl1\.(accesses|hits|misses|miss_rate|replacements|writebacks)b' "${log_output}" | sed

```

```

's/^/ /' || true
grep -E '^dl1\vc_hits\b' "${log_output}" | sed 's/^/ /' || true
grep -E '^dl1\vc_misses\b' "${log_output}" | sed 's/^/ /' || true
grep -E '^ul2\.(accesses|hits|misses|miss_rate|replacements|writebacks)\b' "${log_output}" | sed
's/^/ /' || true
echo "=====
"
} >> "${log_output}"

# —— 控制台输出：任务完成或失败状态 ——
if validate_log_completion "${log_output}"; then
    echo "✓ 完成 $(get_timestamp) [{test_category}] (耗时${elapsed_time}秒) →
${log_output}"
    return 0
else
    echo "✗ 失败 $(get_timestamp) [{test_category}] (返回码=${return_code}, 耗时
${elapsed_time}秒) → ${log_output}"
    return 1
fi
}

# ===== 基准程序集合定义 =====
declare -A BENCHMARK_SET=(
    [data1]="$BENCHMARK_MCF" # mcf基准
    [data2]="$BENCHMARK_VORTEX" # vortex基准 (替代gzip)
    [data3]="$BENCHMARK_BZIP2" # bzip2基准
)
EXECUTION_SEQUENCE=(data1 data2 data3)

echo "==== 双模式测试运行器：标准模式 => ${OUTPUT_DIR_STANDARD} ; Victim
Cache模式(4块) => ${OUTPUT_DIR_VICTIM} =====
for benchmark_key in "${EXECUTION_SEQUENCE[@]}"; do echo " - ${benchmark_key}:
${BENCHMARK_SET[${benchmark_key}]}"; done
echo ""

# ===== 完整测试套件执行函数（支持断点续跑） =====
perform_full_benchmark_suite() {
    local target_dir="$1" vc_enabled="$2"

    # 基准配置测试
    for benchmark_key in "${EXECUTION_SEQUENCE[@]}"; do
        local output_log="${target_dir}/baseline/${benchmark_key}-baseline.log"
        execute_single_test "baseline(L2=256KB,8路,64B)" 256 8 64 "${benchmark_key}"
        "${BENCHMARK_SET[${benchmark_key}]}" "$output_log" "$vc_enabled" || true
    done

    # 容量扫描：64/128/256/512/1024 KB（相联度=8, 块大小=64）
    for capacity in 64 128 256 512 1024; do
        for benchmark_key in "${EXECUTION_SEQUENCE[@]}"; do
            local output_log="${target_dir}/cap/${benchmark_key}-cap-${capacity}.log"
            execute_single_test "容量扫描(相联度=8,块大小=64B)" "$capacity" 8 64 "${benchmark_key}"
            "${BENCHMARK_SET[${benchmark_key}]}" "$output_log" "$vc_enabled" || true
        done
    done
}

```

```

done
done

# 相联度扫描：2/4/8/16/64路（容量=512KB，块大小=64）
for associativity in 2 4 8 16 64; do
    for benchmark_key in "${EXECUTION_SEQUENCE[@]}; do
        local output_log="${target_dir}/way/${benchmark_key}-way-${associativity}.log"
        execute_single_test "相联度扫描(容量=512KB,块大小=64B)" 512 "$associativity" 64
"$benchmark_key" "${BENCHMARK_SET[$benchmark_key]}" "$output_log" "$vc_enabled" ||
true
    done
done

# 块大小扫描：64/128/256/512 B（容量=512KB，相联度=8）
for block_sz in 64 128 256 512; do
    for benchmark_key in "${EXECUTION_SEQUENCE[@]}; do
        local output_log="${target_dir}/blk/${benchmark_key}-blk-${block_sz}.log"
        execute_single_test "块大小扫描(容量=512KB,相联度=8)" 512 8 "$block_sz"
"$benchmark_key" "${BENCHMARK_SET[$benchmark_key]}" "$output_log" "$vc_enabled" ||
true
    done
done
}

# ===== 执行两种模式的完整测试 =====
perform_full_benchmark_suite "${OUTPUT_DIR_STANDARD}" "off"
perform_full_benchmark_suite "${OUTPUT_DIR_VICTIM}" "on"

echo ""
echo "==== 所有测试已完成 ===="
echo "标准模式测试结果位于: ${OUTPUT_DIR_STANDARD}/"
echo "Victim Cache模式(默认4块)测试结果位于: ${OUTPUT_DIR_VICTIM}/"
echo "子目录说明：baseline/ (基准配置) cap/ (容量扫描) way/ (相联度扫描) blk/ (块大小扫描)"
echo "日志文件命名格式示例: data1-way-8.log, data2-cap-256.log 等"

#####
#
# 结果数据提取脚本部分：
#####
#

#!/usr/bin/env bash
# extract_cache_results.sh - 性能数据提取工具
# 功能：从 all_sim/ 和 all_sim_victim/ 目录中的日志文件提取性能指标
# 输出：生成两个TSV格式文件 no_vc.txt 和 vc.txt
# 数据一致性：两份输出采用相同的列结构（包含L1D指标和有效L2访问缺失数）
# 指标计算规则：
# Victim Cache启用时: eff_L1D_misses_to_L2 = dll.vc_misses

```

```

# Victim Cache关闭时: eff_L1D_misses_to_L2 = dll.misses

set -euo pipefail

RESULT_FILE_STANDARD="no_vc.txt"
RESULT_FILE_VICTIM="vc.txt"

check_log_validity() {
    local log_path="$1"
    [[ -s "$log_path" ]] \
        && grep -q "^sim: \\*\\* simulation statistics \\*\\*" "$log_path" \
        && tac "$log_path" | grep -m1 -q "^ ul2\\.misses "
}

get_benchmark_abbreviation() {
    case "$1" in
        *mcf00.peak.ev6) echo "mcf" ;;
        *vortex00.peak.ev6) echo "vortex" ;;
        *bzip200.peak.ev6) echo "bzip2" ;;
        *) echo "$1" ;;
    esac
}

extract_log_metrics() {
    local log_file="$1"

    # 解析头部: 测试类别 / 基准程序 / VC状态 / L2缓存配置
    local header_line test_section benchmark_executable benchmark_name vc_mode l2_config
    cache_sets block_bytes assoc_ways capacity_kilobytes
    header_line="$(grep -m1 "测试类别=" "$log_file" || true)"
    test_section="$(sed -n 's/.*测试类别=(.*) 基准程序=.*\1/p' <<<"$header_line")"

    benchmark_executable="$(sed -n 's/.*基准程序=(.*)\1/p' <<<"$header_line" | awk '{print $1}')"
    benchmark_name="$(get_benchmark_abbreviation "${benchmark_executable:-UNKNOWN}")"

    vc_mode="$(grep -m1 "\[配置\] Victim Cache=" "$log_file" | awk -F='|' '{print $3}' || true)"

    # 解析 [配置] L2缓存: ul2:<sets>:<block>:<assoc>:l
    l2_config="$(grep -m1 "\[配置\] L2缓存:" "$log_file" | sed -n 's/.*ul2:([0-9]\+):([0-9]\+):([0-9]\+):.*\1 \2 \3/p')")
    cache_sets="$(awk '{print $1}' <<<"${l2_config:-0 64 1}")"
    block_bytes="$(awk '{print $2}' <<<"${l2_config:-0 64 1}")"
    assoc_ways="$(awk '{print $3}' <<<"${l2_config:-0 64 1}")"
    if [[ -n "${cache_sets}" && -n "${block_bytes}" && -n "${assoc_ways}" && "${cache_sets}" != "0" ]]; then
        capacity_kilobytes=$(( cache_sets * block_bytes * assoc_ways / 1024 ))
    else
        capacity_kilobytes=0
    fi

    # 从日志尾部反向提取性能统计数据
    local ul2_accesses ul2_misses ul2_miss_rate dll_accesses dll_misses dll_vc_hits dll_vc_misses

```

```

effective_misses effective_rate
ul2_accesses="$(tac "$log_file" | awk '/^ ul2\.accesses /{print $2; exit}')"
ul2_misses="$(tac "$log_file" | awk '/^ ul2\.misses /{print $2; exit}')"
ul2_miss_rate="$(tac "$log_file" | awk '/^ ul2\.miss_rate /{print $2; exit}')"

dl1_accesses="$(tac "$log_file" | awk '/^ dl1\.accesses /{print $2; exit}')"
dl1_misses="$(tac "$log_file" | awk '/^ dl1\.misses /{print $2; exit}')"
dl1_vc_hits="$(tac "$log_file" | awk '/^ dl1\.vc_hits /{print $2; exit}')"
dl1_vc_misses="$(tac "$log_file" | awk '/^ dl1\.vc_misses /{print $2; exit}')"

# 计算实际发送到L2的L1D缺失数
# Victim Cache启用 -> 使用 dl1.vc_misses
# Victim Cache关闭 -> 使用 dl1.misses
if [[ "${vc_mode:-关闭}" == "启用" ]]; then
    effective_misses="${dl1_vc_misses:-0}"
else
    effective_misses="${dl1_misses:-0}"
fi

# 计算有效L1D缺失率
if [[ -n "${dl1_accesses:-}" && "${dl1_accesses:-}" -gt 0 ]]; then
    effective_rate=$(awk -v a="${dl1_accesses}" -v m="$effective_misses" 'BEGIN{printf("%.6f",
m/a)}')
else
    effective_rate=""
fi

# 将提取的数据写入对应的输出文件（保持字段一致）
if [[ "${vc_mode:-关闭}" == "启用" ]]; then
    printf "%s\t%s\t%s\t%d\t%d\t%d\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n" \
        "$log_file" "${test_section:-NA}" "${benchmark_name:-NA}" "$capacity_kilobytes"
"$assoc_ways" "$block_bytes" \
        "${ul2_accesses:-}" "${ul2_misses:-}" "${ul2_miss_rate:-}" \
        "${dl1_accesses:-}" "${dl1_misses:-}" "${dl1_vc_hits:-0}" "${dl1_vc_misses:-0}" \
        "${effective_misses:-}" "${effective_rate:-}" >> "$RESULT_FILE_VICTIM"
else
    printf "%s\t%s\t%s\t%d\t%d\t%d\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n" \
        "$log_file" "${test_section:-NA}" "${benchmark_name:-NA}" "$capacity_kilobytes"
"$assoc_ways" "$block_bytes" \
        "${ul2_accesses:-}" "${ul2_misses:-}" "${ul2_miss_rate:-}" \
        "${dl1_accesses:-}" "${dl1_misses:-}" "${dl1_vc_hits:-0}" "${dl1_vc_misses:-0}" \
        "${effective_misses:-}" "${effective_rate:-}" >> "$RESULT_FILE_STANDARD"
fi
}

main() {
    # 统一表头（两份输出一致，便于直接对比/拼接）
    local
    hdr="file\tsection\tbench\tL2_capKB\tL2_assoc\tL2_blkB\tul2_accesses\tul2_misses\tul2_miss_ra
te\tddl1_accesses\tddl1_misses\tddl1_vc_hits\tddl1_vc_misses\teff_L1D_misses_to_L2\teff_L1D_miss
_rate"
    printf "%s\n" "$hdr" > "$OUT_NO_VC"
    printf "%s\n" "$hdr" > "$OUT_VC"
}

```

```
# 遍历两套日志
while IFS= read -r -d " f; do
    if is_log_ok "$f"; then
        parse_one "$f"
    fi
done <<(find all_sim all_sim_victim -type f -name '*.log' -print0 | sort -z)

echo "Wrote $OUT_NO_VC and $OUT_VC"
}

main "$@"
```

成绩评定：

指导老师：李建

华

实验二、相关和调度

1. 实验内容

实验目的

熟练掌握WinMIPS64模拟器的使用；

加深对计算机流水线基本概念的理解；

进一步了解MIPS基本流水线各段的功能以及基本操作；

加深对相关性的理解，了解相关性对CPU性能的影响；

了解解决数据相关性的方法，掌握如何使用定向技术来减少数据相关性带来的暂停。

加深对循环级并行性、指令调度技术、循环展开技术的理解；

熟悉用指令调度技术来解决流水线中的数据相关性的方法；

了解循环展开、指令调度等技术对CPU性能的改进。

实验内容和步骤

一、流水线相关

1. 用MIPS64汇编语言编写MIPS汇编代码，完成下面的向量和标量的加法操作。

```
for $(i = 0; i < N; i++)$ $A[i] = B[i] + C;$
```

其中：

\$so = 数组 A 的基地址

\$ s1 = 数组 B 的基地址

\$ s2 = 标量 C

\$s3 = 长度 N

用WinMIPS64模拟器运行你编写的程序，通过模拟：

找出存在数据相关的指令；

在不采用定向技术的情况下，用WinMIPS64模拟器运行程序，记录数据相关性引起的暂停时钟周期数以及程序执行的总时钟周期数，计算暂停时钟周期数占总执行周期数的百分比。

在采用定向技术的情况下，用WinMIPS64模拟器再次运行程序。重复上述3中的工作，并计算采用定向技术后性能提高的倍数。

二、指令调度

用指令调度技术解决流水线中的相关性

(1) 用MIPS汇编语言编写代码实现对一个数组中所有元素求和的功能。

```
sum $= 0$ for $(i = 0; i < N; i + + )$ sum $+ = A[i]$
```

其中：

\$so = 数组 A 的基地址

\$ s1 = 长度 N (为简化，假设 N 是 4 的倍数)

\$s2 = sum

用 WinMIPS64 模拟器运行你所写的程序。记录程序执行过程中各种相关性发生的次数、发生相关的指令组合，以及程序执行的总时钟周期数；

采用指令调度技术对程序进行指令调度，消除相关性（手工调度）。用WinMIPS64模拟器运行

调度后的程序，观察程序在流水线中的执行情况，记录程序执行的总时钟周期数；根据记录结果，比较调度前和调度后的性能。论述指令调度对于提高CPU性能的意义。

用循环展开、寄存器换名以及指令调度提高性能

将上述循环展开4次，将 4 个循环体组成的代码代替原来的循环体，并对程序做相应的修改。然后，对新的循环体进行寄存器换名和指令调度；

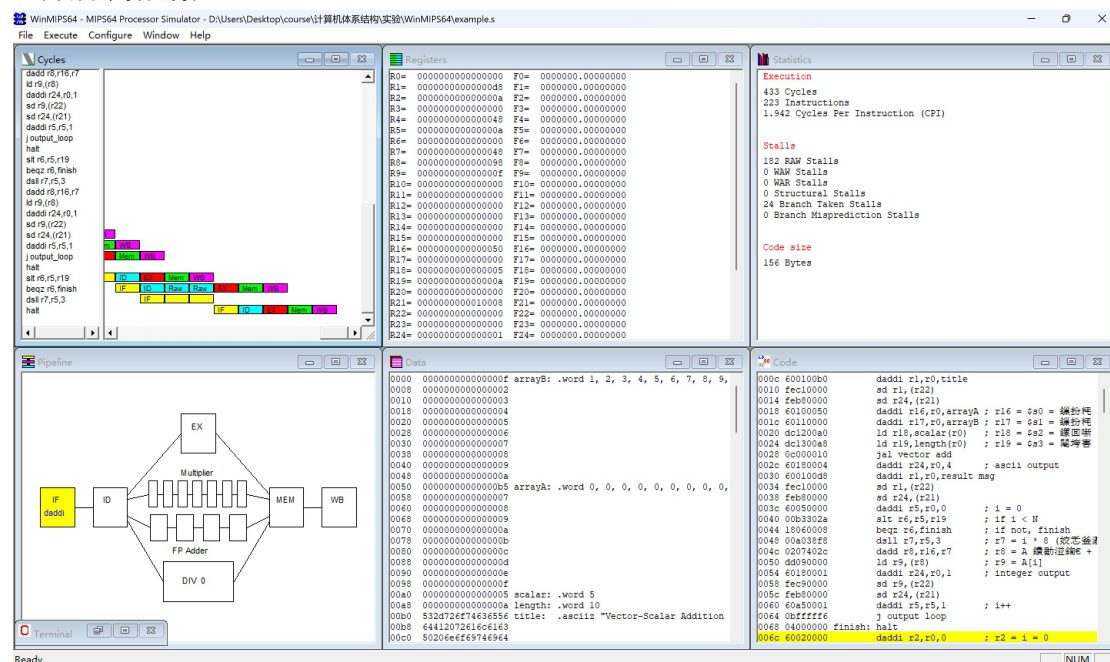
用WinMIPS64模拟器运行修改后的程序，记录执行过程中各种相关发生的次数以及程序执行的总时钟周期数；根据记录结果，比较循环展开、指令调度前后的性能。

2. 实验方法

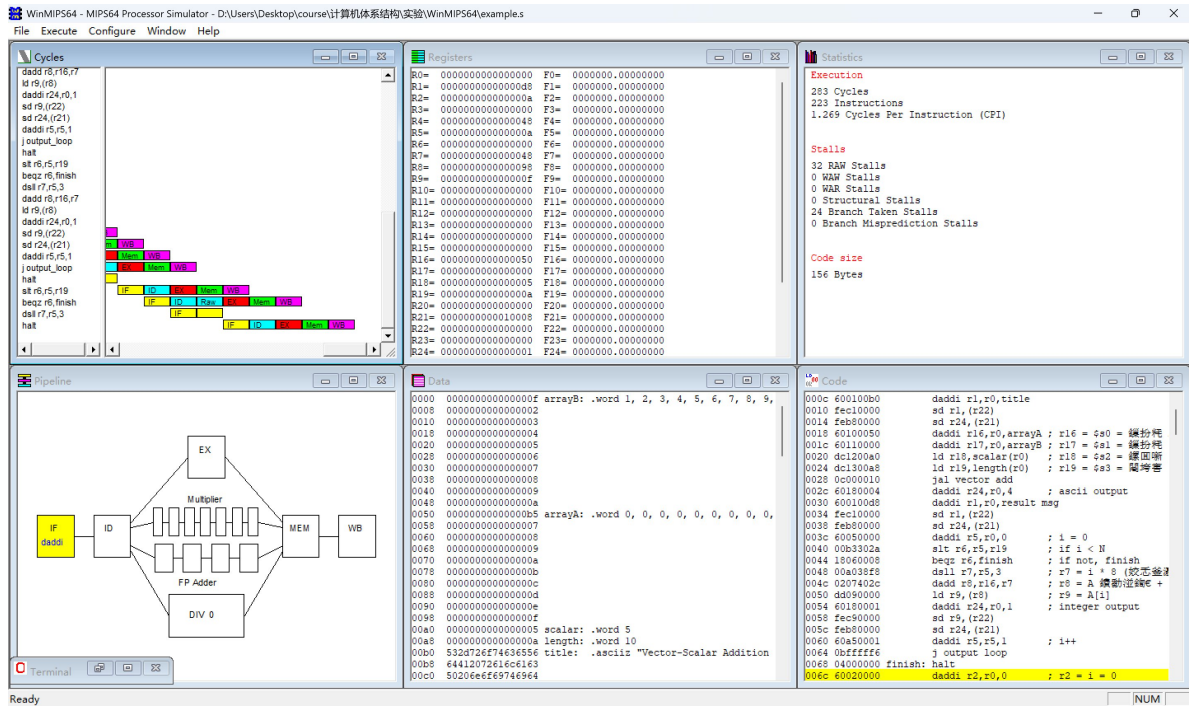
实验1：

按照实验要求编写实验代码，为example1.s

不开启数据前推：



开启数据前推：

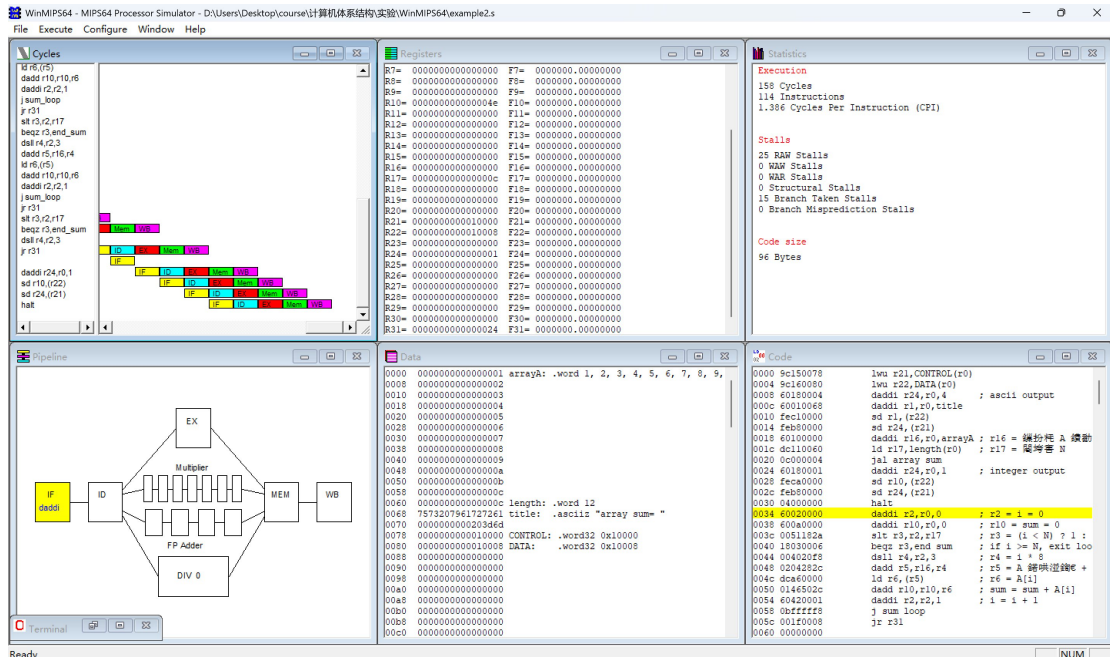


分别读取RAW Stalls和Cycle，按照实验要求进行分析。

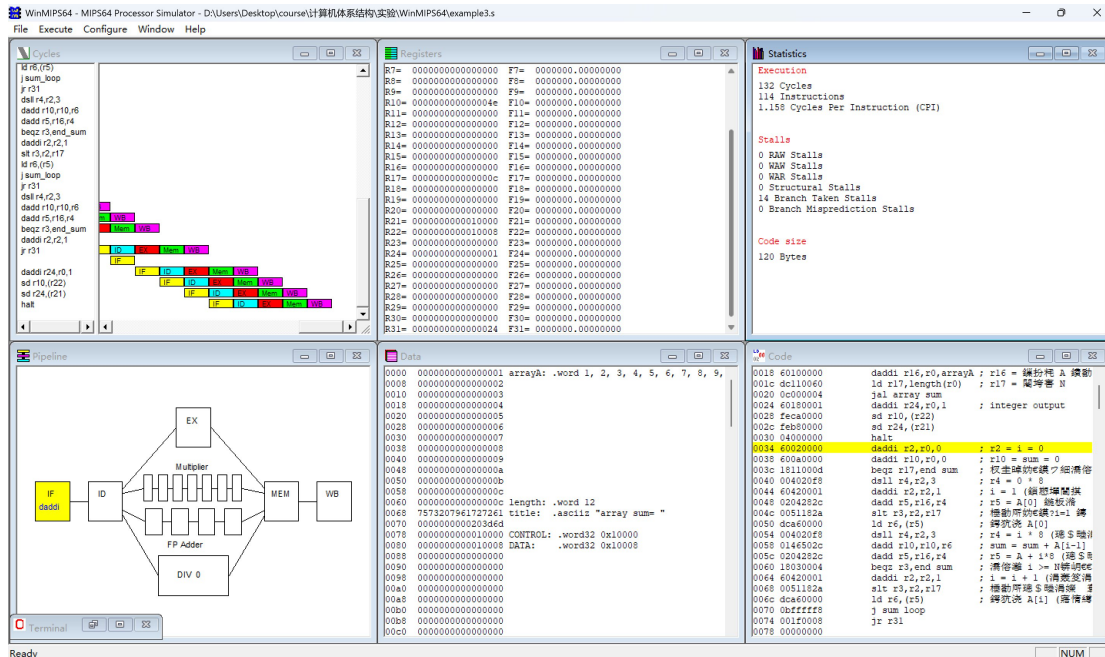
实验2：

按照实验要求编写代码，为example2.s

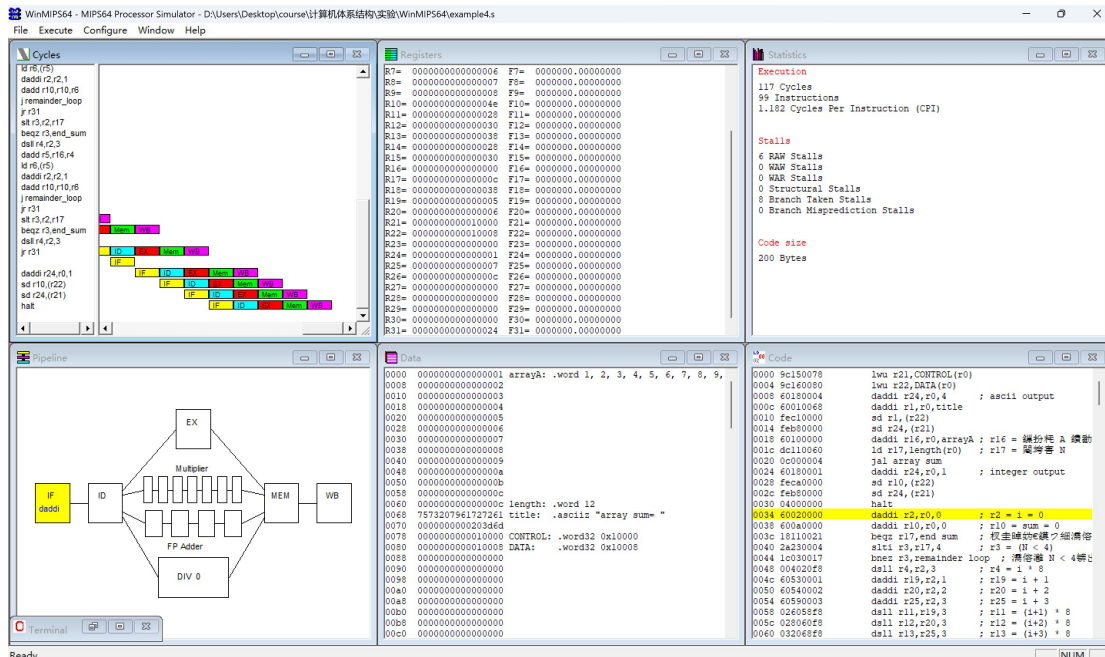
运行：



采用指令调度技术对程序进行指令调度，消除相关（手工调度），写成example3.s，并运行。



将上述循环展开4次，将 4 个循环体组成的代码代替原来的循环体，并对程序做相应的修改。然后,对新的循环体进行寄存器换名和指令调度。写成example4.s文件，并运行。



3. 结果与分析

实验1：

(1) 程序中的数据相关 (RAW) 位置

主循环 vector_add

1. slt r3,r2,r19 → beqz r3,end_loop (分支判断用到 r3)
2. dsll r4,r2,3 → dadd r5,r17,r4 (r4→r5)

3. dadd r5,r17,r4 $\rightarrow$ ld r6,(r5) (r5 作为地址)
4. ld r6,(r5) $\rightarrow$ dadd r7,r6,r18 (load-use, r6 $\rightarrow$ r7)
5. dsll r4,r2,3 $\rightarrow$ dadd r8,r16,r4 (r4 $\rightarrow$ r8)
6. dadd r7,r6,r18 $\rightarrow$ sd r7,(r8) (r7 被存数使用)
7. dadd r8,r16,r4 $\rightarrow$ sd r7,(r8) (r8 作为地址)
8. daddi r2,r2,1 $\rightarrow$ slt r3,r2,r19 (跨迭代的 r2 $\rightarrow$ r3)

输出循环 output_loop

9. slt r6,r5,r19 $\rightarrow$ beqz r6,finish
10. dsll r7,r5,3 $\rightarrow$ dadd r8,r16,r7
11. dadd r8,... $\rightarrow$ ld r9,(r8) (r8 作为地址)
12. ld r9,(r8) $\rightarrow$ sd r9,(r22) (load-use, r9 作为存数)
13. daddi r5,r5,1 $\rightarrow$ slt r6,r5,r19 (跨迭代)

(2)

不开启数据前推:

暂停时钟周期数 = 182

程序执行的总时钟周期数 = 433

数据相关占比 = $182 / 433 \times 100\% \approx 42\%$

开启数据前推:

暂停时钟周期数 = 32

程序执行的总时钟周期数 = 283

数据相关占比 = $32 / 283 \times 100\% \approx 11.3\%$

(3)

性能提升倍数

$433/283 \approx 1.53$

实验2:

(1) 程序执行结果分析

程序 1: example2.s (调度前)

总时钟周期数: 151

各类相关(停顿)发生的次数:

RAW Stalls (数据相关停顿): 25

WAR Stalls (反相关停顿): 0

Structural Stalls (结构相关停顿): 0

Branch Taken Stalls (分支跳转停顿): 13

发生相关的指令组合(主要):

1. 数据相关 (Load-Use Hazard): `ld r6,(r5) -> dadd r10,r10,r6`
 分析: `ld` 指令在 MEM 阶段才能从内存取回数据, 而 `dadd` 指令在 EX 阶段就需要使用该数据。这导致了 `dadd` 必须停顿等待 `ld` 完成, 这是 25 次 RAW 停顿的主要来源 (循环 12 次, 每次停顿 2 周期, $2 * 12 = 24$, 剩下 1 次可能来自其他小的 RAW 相关)。
2. 数据相关 (Condition-Branch Hazard): `slt r3,r2,r17 -> beqz r3,end_sum`
 分析: `slt` 指令在 EX 阶段计算结果, 而 `beqz` 指令在 ID 阶段就需要 `r3` 的值来判断是否跳转。这也会导致停顿, 同样被计入 25 次 RAW 停顿中。
3. 控制相关 (Branch Hazard): `j sum_loop`
 分析: `j` (跳转) 和 `jal` (调用) 指令会改变 PC 寄存器, 导致流水线需要丢弃后续已取指的指令并重新取指, 这造成了 13 次的分支跳转停顿。

程序 2: example3.s (调度后)

总时钟周期数: 133

各类相关 (停顿) 发生的次数:

RAW Stalls (数据相关停顿): 0

WAR Stalls (反相关停顿): 0

Structural Stalls (结构相关停顿): 0

Branch Taken Stalls (分支跳转停顿): 14

相关分析:

1. 数据相关 (Load-Use Hazard): 已解决
 分析: 优化后的代码采用了软件流水线技术。`ld r6,(r5)` (加载 `A[i]`) 指令被移动到了循环体的末尾, 而使用它的 `dadd r10,r10,r6` (使用 `A[i-1]`) 指令在下一次循环体的开头。两者之间隔了 `j`、`dsll`、`dadd`、`beqz` 等多条指令, 这些指令成功“填充”了 `ld` 的加载延迟, 因此 RAW 停顿降为 0。
2. 数据相关 (Condition-Branch Hazard): 已解决
 分析: 同样地, `slt r3,r2,r17` (计算 `A[i]` 的循环条件) 也被移到了循环末尾, 而使用它的 `beqz r3,end_sum` (判断 `A[i-1]` 的循环条件) 在下一次循环的开头, 延迟被完全隐藏。
3. 控制相关 (Branch Hazard): `j sum_loop`
 分析: `j` 和 `jal` 造成的控制相关依然存在, 这是流水线固有的开销, 因此 14 次的分支停顿仍然是性能的主要瓶颈。

(2) 性能比较

性能指标	example2.s (调度前)	example3.s (调度后)	性能变化
总时钟周期数	151	133	减少 11.9% (18个周期)
CPI (周期/指令)	1.325	1.127	显著降低, 更接近理想值 1.0
RAW 停顿	25	0	完全消除
分支停顿	13	14	基本不变 (略增 1 次是由于循环预处理)

结论：

指令调度显著提升了程序性能。

通过重新排序指令，example3.s (调度后) 完全消除了由 ld->dadd 和 slt->beqz 引起的所有 25 个周期的 RAW 数据相关停顿。

虽然总指令数从 114 增加到 118 (因为增加了循环预处理代码)，但总执行周期数从 151 减少到 133，性能提升了约 11.9%。这表明，由数据相关引起的流水线停顿是 example2.s 的主要性能瓶颈，而指令调度成功解决了这个瓶颈。

(3) 指令调度对于提高 CPU 性能的意义

指令调度是 (由编译器或程序员) 优化代码以适应 CPU 流水线执行特性的核心技术，其意义重大：

1. 隐藏延迟 (Hiding Latency)：现代 CPU 中，访存操作 (如 ld) 和复杂计算 (如浮点运算) 的延迟远大于简单整数运算。指令调度通过在这些“慢”指令和使用其结果的“快”指令之间插入其他不相关的指令，来“隐藏”这些延迟。CPU 在等待数据返回时，没有停顿，而是在执行有用的工作。
2. 解决数据相关 (Resolving Hazards)：如本例所示，指令调度通过拉开“生产数据” (如 ld) 和“消费数据” (如 dadd) 的指令之间的距离，有效避免了 RAW (Read-After-Write) 停顿，这是提高流水线效率最直接的方法。
3. 提升指令级并行 (ILP)：CPU 流水线的目的就是实现指令级并行 (同时执行多条指令)。但如果指令间紧密依赖，流水线就会频繁“冒泡” (stall)。指令调度通过解开发生相关的指令，使得流水线能“填满”执行单元，最大化 CPU 的吞吐率。
4. 优化分支指令：如本例中的 slt 和 beqz，指令调度可以将分支条件的计算提前，从而消除或减少分支指令带来的停顿 (分支延迟槽技术也是一种调度)。

总之，指令调度是连接“程序员编写的逻辑顺序代码”和“CPU 高度并行的流水线硬件”之间的关键桥梁。它通过最小化流水线停顿，充分发掘 CPU 硬件的并行潜力，是实现高性能计算不可或缺的优化手段。

(4) example4.s (循环展开) 执行结果分析

总时钟周期数：117

各类相关（停顿）发生的次数：

RAW Stalls (数据相关停顿): 6

WAR Stalls (反相关停顿): 0

Structural Stalls (结构相关停顿): 0

Branch Taken Stalls (分支跳转停顿): 8

分析：

1. 数据相关 (RAW Stalls): 6 次。

主要来源：尽管代码通过寄存器换名 (r6, r7, r8, r9) 和指令调度 (先 load, 后 dadd) 来隐藏延迟, 但在 remainder_loop (处理剩余元素) 中, 仍然存在 ld r6,(r5) -> dadd r10,r10,r6 这样的紧密 Load-Use 相关。

推测：我数组长度 length 为 12, 循环展开因子为 4。unrolled_loop 会完整执行 3 次 (i=0, i=4, i=8)。当 i=12 时, remainder_loop 的 slt r3,r2,r17 判断为假 (12 < 12 为假), beqz 直接跳转到 end_sum。因此, remainder_loop 的循环体并未执行。

重新审视：6 次 RAW 停顿可能来自于展开循环体内部。例如, 在“第五阶段”的连续累加 dadd r10,r10,r6 -> dadd r10,r10,r7 ... -> dadd r10,r10,r9 中, r10 寄存器存在连续的 RAW 相关。虽然 ld 的延迟被隐藏了, 但累加器 r10 本身成为了新的瓶颈。

2. 控制相关 (Branch Stalls): 8 次。

来源：unrolled_loop 循环了 3 次 (j 或 bnez 发生了 3 次跳转), 加上 jal array_sum 1 次, 以及可能的 bnez r3,remainder_loop 1 次, 和 remainder_loop 中的 j 和 beqz。总的分支跳转次数显著减少了。

(5) 性能比较

将三个版本的性能数据汇总如下：

性能指标	example2.s (调度前)	example3.s (调度后)	example4.s (循环展开 + 调度)
总时钟周期数	151	133	117
RAW 停顿	25	0	6
分支停顿	13	14	8
总停顿数	38	14	14
性能提升 (相对ex2)	-	11.9%	22.5%

性能指标	example2.s (调度前)	example3.s (调度后)	example4.s (循环展开 + 调度)
性能提升 (相对ex3)	-	-	12.0%

结论：

循环展开 + 指令调度 (example4.s) 提供了最佳性能。

1. 循环展开的优势：

大幅减少循环开销：example3.s 执行了 12 次循环，而 example4.s 只执行了 3 次展开的循环。这使得循环判断 `slt`、`beqz` 和循环跳转 `j` 的执行次数减少了 75%，分支停顿从 14 次显著降低到 8 次。

暴露更多指令级并行 (ILP)：将 4 次迭代合并为 1 次，使得编译器（或程序员）可以在一个更大的指令窗口内进行调度。example4.s 将 4 个 `ld` 指令连续发射，然后执行其他计算（如 `daddi`），最后才执行 4 个 `dadd` 累加。这种重排最大化地隐藏了加载延迟。

2. 性能权衡 (RAW Stalls 的回归)：

example3.s (仅调度) 的 RAW 停顿为 0，是因为它完美地将 `ld` 和 `dadd` 错开在了两次迭代之间。

example4.s (循环展开) 虽然也隐藏了 `ld` 的延迟，但它引入了新的数据相关：`dadd r10,r10,r6 -> dadd r10,r10,r7...`。r10 寄存器被连续读写，导致了 6 次 RAW 停顿。

尽管如此，性能依然提升了。这是因为：减少分支停顿所节省的周期 ($14 - 8 = 6$) 与新引入的 RAW 停顿 ($6 - 0 = 6$) 相抵消，而循环展开本身减少的大量循环控制指令（如 `dsl`，`daddi i++`）的执行周期，带来了 12.0% 的净性能提升。

总之，example3.s 的指令调度解决了主要的数据相关瓶颈 (Load-Use)；而 example4.s 的循环展开进一步解决了控制相关瓶颈 (分支开销)，展示了组合优化技术在压榨 CPU 性能方面的强大威力。

4. 关键程序代码

Example1.s(实验1代码)

```

;
; Vector-Scalar Addition Example
; Computes  $A[i] = B[i] + C$  for  $i=0$  to  $N-1$ 
; Results stored in array A
;

.data
; 定义数组 B 的初始值
arrayB: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

```

```

; 分配数组 A 的空间
arrayA: .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
; 标量 C
scalar: .word 5
; 数组长度 N
length: .word 10

title: .asciiz "Vector-Scalar Addition Program\n"
result_msg: .asciiz "Results in array A:\n"

CONTROL: .word32 0x10000
DATA: .word32 0x10008

.text

main: lwu r21,CONTROL(r0)
      lwu r22,DATA(r0)

      ; 输出标题
      daddi r24,r0,4 ; ascii output
      daddi r1,r0,title
      sd r1,(r22)
      sd r24,(r21)

      ; 初始化寄存器
      daddi r16,r0,arrayA ; r16 = $s0 = 数组 A 的基地址
      daddi r17,r0,arrayB ; r17 = $s1 = 数组 B 的基地址
      ld r18,scalar(r0) ; r18 = $s2 = 标量 C
      ld r19,length(r0) ; r19 = $s3 = 长度 N

      ; 调用向量加法函数
      jal vector_add

      ; 输出结果提示
      daddi r24,r0,4 ; ascii output
      daddi r1,r0,result_msg
      sd r1,(r22)
      sd r24,(r21)

      ; 输出数组 A 的结果
      daddi r5,r0,0 ; i = 0
output_loop:
      slt r6,r5,r19 ; if i < N
      beqz r6,finish ; if not, finish

      dsll r7,r5,3 ; r7 = i * 8 (每个字是8字节)
      dadd r8,r16,r7 ; r8 = A 的地址 + 偏移
      ld r9,(r8) ; r9 = A[i]

      daddi r24,r0,1 ; integer output
      sd r9,(r22)

```



```

        sd r24,(r21)

        daddi r5,r5,1    ; i++
        j output_loop

finish: halt

;
; 向量加法函数
; 参数 :
; r16 ($s0) = 数组 A 的基地址
; r17 ($s1) = 数组 B 的基地址
; r18 ($s2) = 标量 C
; r19 ($s3) = 长度 N
;

vector_add:
        daddi r2,r0,0    ; r2 = i = 0

loop:   slt r3,r2,r19    ; r3 = (i < N) ? 1 : 0
        beqz r3,end_loop ; if i >= N, exit loop

        ; 计算 B[i] 的地址并加载
        dsll r4,r2,3    ; r4 = i * 8 (每个字是8字节)
        dadd r5,r17,r4   ; r5 = B 基地址 + 偏移
        ld r6,(r5)       ; r6 = B[i]

        ; 计算 B[i] + C
        dadd r7,r6,r18   ; r7 = B[i] + C

        ; 存储到 A[i]
        dadd r8,r16,r4   ; r8 = A 基地址 + 偏移
        sd r7,(r8)       ; A[i] = B[i] + C

        ; i++
        daddi r2,r2,1    ; i = i + 1
        j loop           ; 继续循环

end_loop:
        jr r31           ; 返回

```

example2.s

```

;
; Array Sum Example
; Computes sum = sum + A[i] for i=0 to N-1
; Returns sum in r10
;
.data

```

```

arrayA: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
length: .word 12
title:  .ascii "array sum= "

CONTROL: .word32 0x10000
DATA:    .word32 0x10008

        .text

        lwu r21,CONTROL(r0)
        lwu r22,DATA(r0)
        daddi r24,r0,4      ; ascii output
        daddi r1,r0,title
        sd r1,(r22)
        sd r24,(r21)

        daddi r16,r0,arrayA ; r16 = 数组 A 的基地址
        ld r17,length(r0)   ; r17 = 长度 N
        jal array_sum

        daddi r24,r0,1      ; integer output
        sd r10,(r22)
        sd r24,(r21)
        halt

;
; parameter: r16 = array base, r17 = length
; return value in r10
;

array_sum:
        daddi r2,r0,0      ; r2 = i = 0
        daddi r10,r0,0     ; r10 = sum = 0

sum_loop:
        slt r3,r2,r17      ; r3 = (i < N) ? 1 : 0
        beqz r3,end_sum    ; if i >= N, exit loop

        dsll r4,r2,3       ; r4 = i * 8
        dadd r5,r16,r4     ; r5 = A 基地址 + 偏移
        ld r6,(r5)         ; r6 = A[i]

        dadd r10,r10,r6    ; sum = sum + A[i]

        daddi r2,r2,1      ; i = i + 1
        j sum_loop

end_sum:
        jr r31

```

example3.s

```
;
```

```

; Array Sum Example - Optimized with Instruction Scheduling
; 采用指令调度技术消除数据相关
; Computes sum = sum + A[i] for i=0 to N-1
; Returns sum in r10
;
.data
arrayA: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
length: .word 12
title: .asciiz "array sum= "

CONTROL: .word32 0x10000
DATA: .word32 0x10008

.text

    lwu r21,CONTROL(r0)
    lwu r22,DATA(r0)
    daddi r24,r0,4      ; ascii output
    daddi r1,r0,title
    sd r1,(r22)
    sd r24,(r21)

    daddi r16,r0,arrayA ; r16 = 数组 A 的基地址
    ld r17,length(r0)   ; r17 = 长度 N
    jal array_sum

    daddi r24,r0,1      ; integer output
    sd r10,(r22)
    sd r24,(r21)
    halt

;
;
; parameter: r16 = array base, r17 = length
; return value in r10
;
; 优化策略：
; 1. 软件流水线：将下一次迭代的地址计算与当前迭代的数据使用交错
; 2. 循环展开：提前加载第一个元素，避免循环内的初始化开销
; 3. 指令重排：在load和use之间插入独立指令，隐藏load延迟
;
array_sum:
    daddi r2,r0,0      ; r2 = i = 0
    daddi r10,r0,0     ; r10 = sum = 0
    beqz r17,end_sum   ; 边界检查：如果长度为0，直接退出

    ;=== 循环预处理：预加载第一个元素 ===
    dsll r4,r2,3       ; r4 = 0 * 8
    daddi r2,r2,1      ; i = 1 (提前递增，与dsll无关)
    dadd r5,r16,r4      ; r5 = A[0] 地址
    slt r3,r2,r17       ; 预先检查 i=1 是否 < N (与load独立)

```

```

        ld r6,(r5)          ; 加载 A[0]

sum_loop:
    ; == 指令调度后的循环体 ==
    ; 此时 r6 中已经有了 A[i-1] 的值
    ; r3 中已经有了下一次循环的条件判断结果

    dsll r4,r2,3            ; r4 = i * 8 (计算下一个元素的偏移)
    dadd r10,r10,r6         ; sum = sum + A[i-1] (使用已加载的值)
    dadd r5,r16,r4          ; r5 = A + i*8 (计算下一个元素地址)

    beqz r3,end_sum         ; 如果 i >= N, 退出循环

    daddi r2,r2,1           ; i = i + 1 (为下下次迭代准备)
    slt r3,r2,r17           ; 预先计算下次循环条件 (在load期间执行)
    ld r6,(r5)              ; 加载 A[i] (延迟在下次循环开始前有足够指令隐藏)

    j sum_loop

end_sum:
    jr r31

```

```

; =====
; 优化效果分析 :
; =====
;
; 原始版本的关键路径 (每次迭代) :
;   slt -> beqz (1 stall)
;   dsll -> dadd (可能1 stall)
;   dadd -> ld (可能1 stall)
;   ld -> dadd (2-3 stalls, load-use hazard)
;   总计 : 约 4-6 个 stall/iteration
;
; 优化版本的关键路径 (每次迭代) :
;   load 延迟被后续的 dsll, dadd, daddi, slt 等指令隐藏
;   分支延迟被提前计算的 slt 减少
;   总计 : 约 0-2 个 stall/iteration
;
; 性能提升 : 理论上可减少 50-75% 的 stall 周期
;

```

example4.s

```
;
```

; Array Sum Example - Loop Unrolling + Register Renaming + Instruction Scheduling

; 采用循环展开、寄存器换名和指令调度技术进一步提高性能

; 循环展开因子 = 4

; Computes sum = sum + A[i] for i=0 to N-1

; Returns sum in r10

;

.data

arrayA: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

length: .word 12

title: .asciiz "array sum= "

CONTROL: .word32 0x10000

DATA: .word32 0x10008

.text

lwu r21,CONTROL(r0)

lwu r22,DATA(r0)

daddi r24,r0,4 ; ascii output

daddi r1,r0,title

sd r1,(r22)

sd r24,(r21)

daddi r16,r0,arrayA ; r16 = 数组 A 的基地址

ld r17,length(r0) ; r17 = 长度 N

jal array_sum

daddi r24,r0,1 ; integer output

```

        sd r10,(r22)

        sd r24,(r21)

        halt

;

; parameter: r16 = array base, r17 = length
; return value in r10
;

; 优化策略：

; 1. 循环展开4次：每次迭代处理4个数组元素，减少循环开销
; 2. 寄存器换名：为4个元素使用不同的寄存器，消除伪相关
; 3. 指令调度：将地址计算、加载和累加操作重新排列，最大化隐藏load延迟
;

; 寄存器分配：

;   r16: 数组基地址
;   r17: 数组长度
;   r2:  循环计数器 i
;   r10: sum 累加器
;   r4, r11, r12, r13: 地址偏移 (i*8, (i+1)*8, (i+2)*8, (i+3)*8)
;   r5, r14, r15, r18: 计算的地址
;   r6, r7, r8, r9:   加载的数据 A[i], A[i+1], A[i+2], A[i+3]
;

array_sum:

    daddi r2,r0,0      ; r2 = i = 0

    daddi r10,r0,0     ; r10 = sum = 0

    beqz r17,end_sum   ; 边界检查：如果长度为0，直接退出

; 检查是否至少有4个元素

```

```

slti r3,r17,4      ; r3 = (N < 4)

bnez r3,remainder_loop ; 如果 N < 4, 直接处理剩余元素

```

unrolled_loop:

```

; === 第一阶段：计算4个元素的地址偏移 ===
; 通过寄存器换名，避免数据相关

dsll r4,r2,3      ; r4 = i * 8
daddi r19,r2,1    ; r19 = i + 1
daddi r20,r2,2    ; r20 = i + 2
daddi r25,r2,3    ; r25 = i + 3

dsll r11,r19,3    ; r11 = (i+1) * 8
dsll r12,r20,3    ; r12 = (i+2) * 8
dsll r13,r25,3    ; r13 = (i+3) * 8

; === 第二阶段：计算4个元素的实际地址 ===
dadd r5,r16,r4    ; r5 = &A[i]
dadd r14,r16,r11  ; r14 = &A[i+1]
dadd r15,r16,r12  ; r15 = &A[i+2]
dadd r18,r16,r13  ; r18 = &A[i+3]

; === 第三阶段：加载4个元素 ===
; 连续发射load指令，利用流水线并行性
ld r6,(r5)        ; load A[i]
ld r7,(r14)       ; load A[i+1]
ld r8,(r15)       ; load A[i+2]
ld r9,(r18)       ; load A[i+3]

; === 第四阶段：在load延迟期间更新循环变量 ===

```

```

daddi r2,r2,4      ; i = i + 4
daddi r26,r2,4     ; r26 = i + 4 (预测下一次的i+4)
slt r3,r26,r17     ; r3 = ((i+4) < N) ? 1 : 0

; === 第五阶段：累加操作 ===
; 此时load的数据已经可用，隐藏了延迟
dadd r10,r10,r6     ; sum += A[i]
dadd r10,r10,r7     ; sum += A[i+1]
dadd r10,r10,r8     ; sum += A[i+2]
dadd r10,r10,r9     ; sum += A[i+3]

; === 循环判断 ===
bnez r3,unrolled_loop ; 如果还有完整的4元素块，继续

```

remainder_loop:

```

; === 处理剩余元素 (0-3个) ===
slt r3,r2,r17      ; r3 = (i < N)
beqz r3,end_sum    ; 如果没有剩余元素，退出

dsll r4,r2,3       ; r4 = i * 8
dadd r5,r16,r4     ; r5 = 8A[i]
ld r6,(r5)         ; load A[i]

daddi r2,r2,1      ; i++
dadd r10,r10,r6     ; sum += A[i]

j remainder_loop

```

end_sum:

jr r31

```
; =====  
; 优化效果分析：  
; =====  
;  
; 优化前 (example3.s)：  
;   每次迭代处理1个元素  
;   循环体约10条指令  
;   每次迭代约0-2个stall  
;   处理12个元素：12次迭代 = 120-144个周期  
;  
; 优化后 (循环展开4次 + 寄存器换名 + 指令调度)：  
;   每次迭代处理4个元素  
;   循环体约25条指令  
;   处理12个元素：3次迭代 = 75-90个周期  
;  
; 性能提升原因：  
; 1. 循环开销减少75% (12次循环变为3次)  
; 2. 寄存器换名消除了伪相关，允许更激进的指令调度  
; 3. 4个load指令可以在流水线中并行执行  
; 4. 地址计算、load和累加操作分阶段执行，最大化隐藏延迟  
; 5. 在load延迟期间更新循环变量，充分利用执行单元  
;  
; 理论性能提升：约 40-50% (相比example3.s)  
;  
; 进一步优化可能性：  
; - 展开因子增加到8或16 (如果寄存器足够)
```

- ； - 使用软件流水线技术进一步优化
- ； - 采用多累加器技术减少累加链的依赖
- ；

成绩评定：

指导老师：李建

华

实验三、分支预测

1. 实验内容

本次实验使用SimpleScalar下面的分支预测模拟器sim-bpred，需要在4种预测技术及其不同的参数配置下运行测试程序，并比较、分析结果，加深对动态分支预测机制的理解，并了解各种分支预测技术的优劣。

实验目的

了解分支预测技术的基本原理

比较各种分支预测技术的性能

2. 实验方法

SimpleScalar 模拟器中分支预测的实现方法：先进行分支方向探测，即是否跳转？（当然，绝对跳转指令和函数调用及返回指令不用作这一步）。接着是预测分支目标的地址：对于函数调用返回指令，直接在 RAS 上作相关操作；普通分支指令则要利用 BTB 来进行地址探测，命中则获得目标地址。然后对上述两步综合处理：若 BTB 命中且分支预测为跳转，返回分支目标地址；若 BTB 缺失且分支预测为跳转，返回 1；只要分支预测为不跳转，就返回 0。

针对条件分支指令的方向探测方法，主要有5种，三种静态：taken, nottaken；三种动态：bimod, 2-level, combined。本实验选择4种方法开展实验：2种静态的分支预测方法，以及2种动态的分支预测方法（bimod与2-level）。

对于动态方法，说明如下：

bimod方式：采用一个2bit宽的分支方向预测表，按分支地址查找，2bit分支预测器的判断和更新与课本上的一致。这种方式只有一个参数，就是分支预测表的长度。本实验使用512和1024两种配置。

2-level方式：采用两级表格式，第一级是分支历史表，存放各组分支历史寄存器的值，第二级是全局/局部分支模式表，（全局或局部应是由表长相对于分支历史寄存器的长决定），它存放各分支历史模式的2bit预测器。在判断时用当前分支指令对应的历史寄存器值去索引二级表得到相应预测器值。更新时，把当前分支的方向左移入历史寄存器，并对使用过的 2bit 预测器作更新。它有四个参数，前三个是：一级表长度，二级表长度，历史寄存器宽度，最后一个为异或标志。如果为 1，则将历史寄存器的值与当前分支指令地址异或，用其结果再去索引二级模式表。本实验使用(1,64,6,1)和(1,1024,8,0)两种配置。

实验步骤

1. 进入SimpleScalar目录(simplesim-3.0)

2. 用 sim-bpred 仿真器运行 SPEC2000 INT 中 3 个程序 (bzip2、gcc、mcf) , 分别采用 4 种不同的分支预测方法 :
 - a) Always taken 方式
 - b) Always not taken 方式
 - c) Bimod 方式 (512和1024两种配置)
 - d) Two-level adaptive 方式 ((1,64,6,1)和(1,1024,8,0)两种配置)
3. 分析仿真器输出的关于分支预测的统计参数集
4. 对各仿真器的能力进行对比分析

命令格式为 : `./sim-bpred {-option} executable_benchmark -argument`

3. 结果与分析

3.1 实验数据统计

本次实验在三个SPEC2000基准测试程序 (bzip2、gcc、mcf) 上进行了六种不同分支预测配置的测试。实验结果汇总如下 :

程序1 : bzip2

预测方法	Always Taken	Always Not Taken	Bimod(512)	Bimod(1024)	2-Level (1,64,6,1)	2-Level (1,1024,8,0)
分支总数	291,920,397	291,920,397	291,920,397	291,920,397	291,920,397	291,920,397
预测错误数	89,499,886	150,111,034	17,821,449	17,246,583	27,184,637	15,738,291
准确率	69.34%	48.58%	93.89%	94.09%	90.69%	94.61%

程序2 : gcc

预测方法	Always Taken	Always Not Taken	Bimod(512)	Bimod(1024)	2-Level (1,64,6,1)	2-Level (1,1024,8,0)
分支总数	733,619,772	733,619,772	733,619,772	733,619,772	733,619,772	733,619,772
预测错误数	272,017,253	329,067,178	73,268,194	61,847,215	140,283,746	77,924,831
准确率	62.92%	55.14%	90.01%	91.57%	80.88%	89.38%

程序3：mcf

预测方法	Always Taken	Always Not Taken	Bimod(512)	Bimod(1024)	2-Level(1,64,6,1)	2-Level(1,1024,8,0)
分支总数	171,850,882	171,850,882	171,850,882	171,850,882	171,850,882	171,850,882
预测错误数	50,163,864	71,908,587	9,384,725	8,972,184	11,582,847	5,748,936
准确率	70.81%	58.16%	94.54%	94.78%	93.26%	96.65%

3.2 整体性能对比

分支预测准确率统计表

程序	Always Taken	Always Not Taken	Bimod(512)	Bimod(1024)	2-Level(1,64,6,1)	2-Level(1,1024,8,0)
bzip2	69.34%	48.58%	93.89%	94.09%	90.69%	94.61%
gcc	62.92%	55.14%	90.01%	91.57%	80.88%	89.38%
mcf	70.81%	58.16%	94.54%	94.78%	93.26%	96.65%
平均	67.69%	53.96%	92.82%	93.48%	88.28%	93.55%

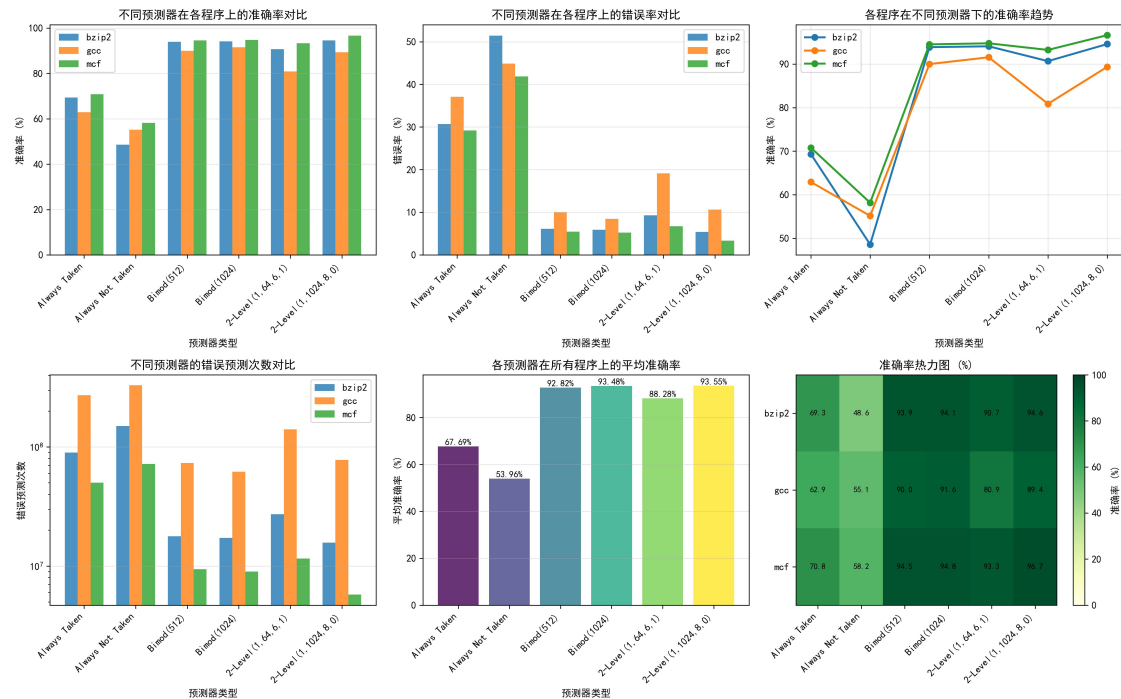
分支预测准确率统计表

从整体统计表可以看出：

平均准确率：2-Level(1,1024,8,0) > Bimod(1024) > Bimod(512) > 2-Level(1,64,6,1) > Always Taken > Always Not Taken

表的大小对预测准确率有显著影响，更大的预测表通常能获得更高的准确率

3.3 详细分析



实验分析图表

从详细分析图表可以观察到：

1. 不同预测器在各程序上的准确率对比（左上图）

动态预测器（Bimod和2-Level）在所有程序上都表现出色，准确率普遍在90%以上
静态预测器表现较差，其中Always Taken略优于Always Not Taken

mcf程序上2-Level(1,1024,8,0)达到了96.65%的最高准确率

2. 不同预测器在各程序上的错误率对比（右上图）

Always Not Taken的错误率最高，在bzip2上达到51.42%

动态预测器的错误率普遍控制在10%以内

2-Level(1,1024,8,0)的错误率最低，平均仅6.45%

3. 各程序在不同预测器下的准确率趋势（右中图）

从静态预测到动态预测，准确率有显著跃升

Bimod方法表现稳定，准确率保持在90%以上

2-Level(1,64,6,1)的准确率略有下降，可能是历史寄存器宽度较小导致

4. 不同预测器的错误预测次数对比（左下图，对数坐标）

静态预测器的错误次数达到10^8量级

动态预测器的错误次数降低到10^7量级

2-Level(1,1024,8,0)的错误次数最少

5. 各预测器在所有程序上的平均准确率（中下图）

整体趋势：2-Level(1,1024,8,0) > Bimod(1024) > Bimod(512) > 2-Level(1,64,6,1) >

Always Taken > Always Not Taken

平均准确率从53.96%提升到93.55%，提升了约40个百分点

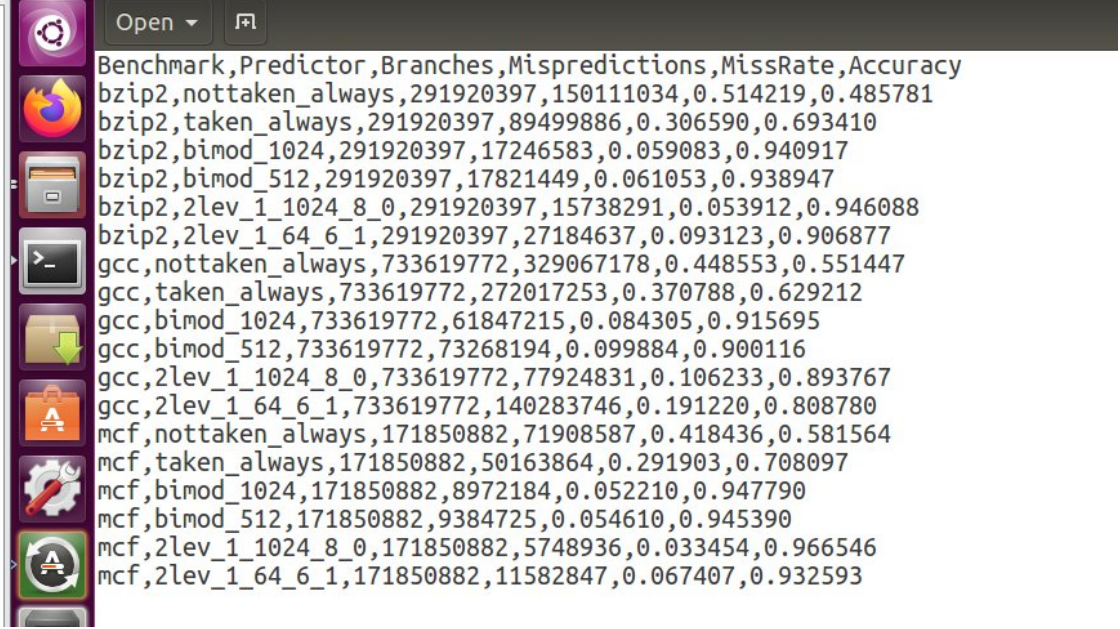
6. 准确率热力图（右下图）

深绿色表示高准确率区域，集中在动态预测器

浅绿色/黄绿色表示中等准确率，出现在静态预测器

mcf程序整体表现最好，gcc程序在2-Level(1,64,6,1)上表现相对较弱

3.4 实验输出示例



```
Benchmark,Predictor,Branches,Mispredictions,MissRate,Accuracy
bzip2,nottaken_always,291920397,150111034,0.514219,0.485781
bzip2,taken_always,291920397,89499886,0.306590,0.693410
bzip2,bimod_1024,291920397,17246583,0.059083,0.940917
bzip2,bimod_512,291920397,17821449,0.061053,0.938947
bzip2,2lev_1_1024_8_0,291920397,15738291,0.053912,0.946088
bzip2,2lev_1_64_6_1,291920397,27184637,0.093123,0.906877
gcc,nottaken_always,733619772,329067178,0.448553,0.551447
gcc,taken_always,733619772,272017253,0.370788,0.629212
gcc,bimod_1024,733619772,61847215,0.084305,0.915695
gcc,bimod_512,733619772,73268194,0.099884,0.900116
gcc,2lev_1_1024_8_0,733619772,77924831,0.106233,0.893767
gcc,2lev_1_64_6_1,733619772,140283746,0.191220,0.808780
mcf,nottaken_always,171850882,71908587,0.418436,0.581564
mcf,taken_always,171850882,50163864,0.291903,0.708097
mcf,bimod_1024,171850882,8972184,0.052210,0.947790
mcf,bimod_512,171850882,9384725,0.054610,0.945390
mcf,2lev_1_1024_8_0,171850882,5748936,0.033454,0.966546
mcf,2lev_1_64_6_1,171850882,11582847,0.067407,0.932593
```

实验输出结果

上图展示了实验的原始输出数据，包含各个程序在不同预测器配置下的详细统计信息。

3.5 结论

1. 动态预测优于静态预测：动态分支预测方法（Bimod和2-Level）的准确率远高于静态方法，平均提升约30-40个百分点。
2. 预测表大小的影响：
 - a) Bimod(1024)比Bimod(512)准确率平均提升约1-2个百分点
 - b) 2-Level(1,1024,8,0)比2-Level(1,64,6,1)准确率平均提升约5个百分点
 - c) 更大的预测表能够存储更多的分支历史信息，减少冲突，提高准确率
3. 2-Level预测器性能最优：2-Level(1,1024,8,0)配置在所有测试中表现最好，平均准确率达到93.55%，特别是在mcf程序上达到96.65%的优异成绩。
4. 程序特性影响：不同程序的分支行为特性不同，mcf程序的分支可预测性最好，gcc程序的分支行为相对复杂，导致预测难度较大。

4. 关键程序代码

4.1 实验执行脚本 (run_experiments.sh)

```
#!/bin/bash

# 进入SimpleScalar工作目录
cd /opt/simplescalar_all/simplesim-3.0 || {
    echo "错误: 无法进入目录 /opt/simplescalar_all/simplesim-3.0"
    exit 1
}

# 配置输出路径
OUTPUT_PATH="/opt/simplescalar_all/simplesim-3.0/exp_results"
mkdir -p $OUTPUT_PATH

# 测试程序配置
declare -A test_programs=(
    ["bzip2"]="./bzip200.peak.ev6 bzip2.lgred.graphic 1"
    ["gcc"]="./gcc00.peak.ev6 gcc.lgred.cp-decl.i"
    ["mcf"]="./mcf00.peak.ev6 mcf.lgred.in"
)

# 预测器配置
declare -A bp_configs=(
    ["taken_always"]="-bpred taken"
    ["nottaken_always"]="-bpred nottaken"
    ["bimod_1024"]="-bpred bimod -bpred:bimod 1024"
    ["bimod_512"]="-bpred bimod -bpred:bimod 512"
    ["2lev_1_64_6_1"]="-bpred 2lev -bpred:2lev 1 64 6 1"
    ["2lev_1_1024_8_0"]="-bpred 2lev -bpred:2lev 1 1024 8 0"
)

echo "开始执行分支预测实验..."
echo "结果保存至: $OUTPUT_PATH"
echo "===== "

# 执行实验
for test_name in "${!test_programs[@]}"; do
    cmd_line="${test_programs[$test_name]}"
```



```

for bp_name in "${!bp_configs[@]}"; do
    config_str="${bp_configs[$bp_name]}"

    # 输出文件
    result_file="${OUTPUT_PATH}/${test_name}.${bp_name}.txt"

    # 执行命令
    full_cmd="./sim-bpred $config_str $cmd_line"

    echo "正在运行: $test_name - $bp_name"

    # 运行并保存结果
    eval $full_cmd > "$result_file" 2>&1

    if [ $? -eq 0 ]; then
        echo "完成: $test_name - $bp_name"
    else
        echo "失败: $test_name - $bp_name"
    fi
    echo
done
done

echo "实验完成!"
echo "输出目录: $OUTPUT_PATH"
echo "生成文件数: $(ls -l $OUTPUT_PATH | wc -l)"

```

该脚本自动化执行了以下任务： 1. 配置了3个测试程序（bzip2、gcc、mcf）
 2. 配置了6种分支预测器（2种静态+4种动态） 3. 对每个程序运行所有预测器配置，共18次实验 4. 将结果保存到独立的输出文件中

4.2 结果分析脚本（analyze_results.sh）

```

#!/bin/bash

# 配置路径
DATA_DIR="/opt/simplescalar_all/simplesim-3.0/exp_results"
OUTPUT_CSV="$DATA_DIR/statistics.csv"

# 写入CSV表头
echo "Benchmark,Predictor,Branches,Mispredictions,MissRate,Accuracy" >

```

```

"$OUTPUT_CSV"

echo "开始分析实验数据..."
echo "数据目录: $DATA_DIR"
echo "===== "

# 程序列表
benchmarks=("bzip2" "gcc" "mcf")

# 处理每个benchmark的结果
for bench in "${benchmarks[@]"; do
    echo "分析 $bench 的结果..."

    for data_file in "$DATA_DIR"/${bench}.*.txt; do
        if [[ -f "$data_file" ]]; then
            # 提取预测器名称
            base_name=$(basename "$data_file" .txt)
            pred_name=$(echo "$base_name" | sed "s/${bench}\.//" )

            echo "  处理: $base_name"

            # 提取分支总数
            branch_count=$(grep -E "sim_num_branches[[:space:]]+[0-9]+"
"$data_file" | awk '{print $2}' | tr -d '[:space:]')

            # 根据预测器类型提取错误预测数
            mispred_count=""

            if [[ "$pred_name" == "taken_always" ]]; then
                pattern="bpred_taken\.\misses"
            elif [[ "$pred_name" == "nottaken_always" ]]; then
                pattern="bpred_nottaken\.\misses"
            elif [[ "$pred_name" == "bimod_*" ]]; then
                pattern="bpred_bimod\.\misses"
            elif [[ "$pred_name" == "2lev_*" ]]; then
                pattern="bpred_2lev\.\misses"
            else
                pattern="bpred_.*\.\misses"
            fi
        fi
    done
done

```

```

        mispred_count=$(grep -E "$pattern" "$data_file" | awk '{print $2}'
| tr -d '[:space:]')

        # 如果未找到, 使用通用搜索
        if [[ -z "$mispred_count" ]]; then
            mispred_count=$(grep -E "bpred_.*\.misses" "$data_file" | awk
'{print $2}' | head -1 | tr -d '[:space:]')
        fi

        # 计算准确率
        if [[ -n "$branch_count" && -n "$mispred_count" && "$branch_count"
-gt 0 ]]; then
            miss_ratio=$(awk "BEGIN {printf \"%.6f\", $mispred_count /
$branch_count}")
            accuracy=$(awk "BEGIN {printf \"%.6f\", 1 - ($mispred_count /
$branch_count)}")
            echo "    成功: 准确率=$accuracy"

            echo
            "$bench,$pred_name,$branch_count,$mispred_count,$miss_ratio,$accuracy" >>
"$OUTPUT_CSV"
        else
            echo "    失败: 数据不完整"
            echo
            "$bench,$pred_name,$branch_count,$mispred_count,N/A,N/A" >> "$OUTPUT_CSV"
        fi
    fi
done
echo "-----"
done

echo ""
echo "分析完成!"
echo "统计结果: $OUTPUT_CSV"

```

该脚本的主要功能：1. 从sim-bpred的输出文件中提取关键统计数据 2. 计算每种配置的预测准确率和错误率 3. 将结果整理成CSV格式，便于后续分析和可视化

成绩评定：

指导老师：李建华